

CHƯƠNG 14. THỊ GIÁC MÁY TÍNH CHUYÊN SÂU SỬ DỤNG MẠNG NƠ-RON TÍCH CHẬP

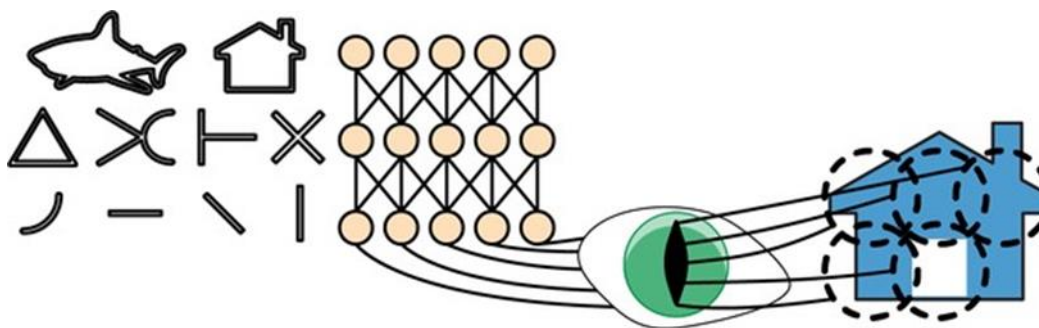
Mặc dù siêu máy tính Deep Blue của IBM đã đánh bại nhà vô địch cờ vua thế giới Garry Kasparov vào năm 1996, nhưng phải đến gần đây máy tính mới có thể thực hiện một cách đáng tin cậy các tác vụ tưởng chừng như đơn giản như phát hiện một chú chó con trong ảnh hoặc nhận dạng lời nói. Tại sao những tác vụ này lại dễ dàng đối với con người chúng ta đến vậy? Câu trả lời nằm ở chỗ nhận thức phần lớn diễn ra bên ngoài phạm vi ý thức của chúng ta, trong các mô-đun thị giác, thính giác và các giác quan khác chuyên biệt trong não bộ. Khi thông tin giác quan đến ý thức của chúng ta, nó đã được trang bị các tính năng cấp cao; ví dụ, khi bạn nhìn vào một bức ảnh về một chú chó con đáng yêu, bạn không thể không nhìn thấy chú chó con, không nhận thấy sự đáng yêu của nó. Bạn cũng không thể giải thích làm thế nào bạn nhận ra một chú chó con đáng yêu; điều đó chỉ đơn giản là hiển nhiên đối với bạn. Do đó, chúng ta không thể tin vào trải nghiệm chủ quan của mình: nhận thức không hề đơn giản chút nào, và để hiểu nó, chúng ta phải xem xét cách các mô-đun giác quan của chúng ta hoạt động.

Mạng nơ-ron tích chập (CNN) ra đời từ nghiên cứu về vỏ não thị giác của não bộ, và chúng đã được sử dụng trong nhận dạng hình ảnh máy tính từ những năm 1980. Trong 10 năm qua, nhờ sự gia tăng sức mạnh tính toán, lượng dữ liệu huấn luyện có sẵn và các thủ thuật được trình bày trong Chương 11 để huấn luyện các mạng sâu, CNN đã đạt được hiệu suất siêu phàm trong một số tác vụ thị giác phức tạp. Chúng cung cấp sức mạnh cho các dịch vụ tìm kiếm hình ảnh, xe tự lái, hệ thống phân loại video tự động và nhiều hơn nữa. Hơn nữa, CNN không chỉ giới hạn trong nhận thức thị giác: chúng còn thành công trong nhiều tác vụ khác, chẳng hạn như nhận dạng giọng nói và xử lý ngôn ngữ tự nhiên. Tuy nhiên, chúng ta sẽ tập trung vào các ứng dụng thị giác trong chương này.

Trong chương này, chúng ta sẽ khám phá nguồn gốc của CNN, các khối xây dựng của chúng trông như thế nào và cách triển khai chúng bằng Keras. Sau đó, chúng ta sẽ thảo luận về một số kiến trúc CNN tốt nhất, cũng như các tác vụ thị giác khác, bao gồm phát hiện đối tượng (phân loại nhiều đối tượng trong một hình ảnh và đặt các hộp bao quanh chúng) và phân đoạn ngữ nghĩa (phân loại từng pixel theo lớp của đối tượng mà nó thuộc về).

Kiến trúc của vỏ não thị giác

David H. Hubel và Torsten Wiesel đã thực hiện một loạt thí nghiệm trên mèo vào năm 1958 và 1959 (và vài năm sau đó trên khỉ), đưa ra những hiểu biết quan trọng về cấu trúc của vỏ não thị giác (các tác giả đã nhận giải Nobel Sinh lý học hoặc Y học năm 1981 cho công trình của họ). Đặc biệt, họ đã chỉ ra rằng nhiều nơ-ron trong vỏ não thị giác có một trường tiếp nhận cục bộ nhỏ, có nghĩa là chúng chỉ phản ứng với các kích thích thị giác nằm trong một vùng giới hạn của trường thị giác (xem Hình 14-1, trong đó các trường tiếp nhận cục bộ của năm nơ-ron được biểu thị bằng các vòng tròn đứt nét). Các trường tiếp nhận của các nơ-ron khác nhau có thể chồng chéo lên nhau, và cùng nhau chúng bao phủ toàn bộ trường thị giác.



Hình 14-1. Các nơ-ron sinh học trong vỏ não thị giác phản ứng với các mẫu cụ thể trong các vùng nhỏ của trường thị giác được gọi là trường tiếp nhận; khi tín hiệu thị giác đi qua các mô-đun não liên tiếp, các nơ-ron phản ứng với các mẫu phức tạp hơn trong các trường tiếp nhận lớn hơn.

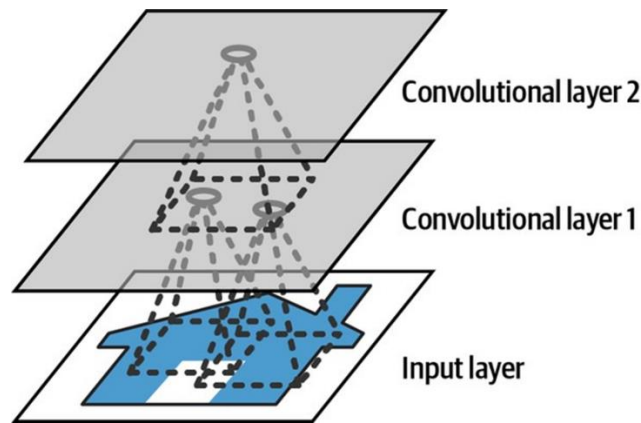
Hơn nữa, các tác giả đã chỉ ra rằng một số nơ-ron chỉ phản ứng với hình ảnh các đường ngang, trong khi những nơ-ron khác chỉ phản ứng với các đường có hướng khác nhau (hai nơ-ron có thể có cùng trường tiếp nhận nhưng phản ứng với các hướng đường khác nhau). Họ cũng nhận thấy rằng một số nơ-ron có trường tiếp nhận lớn hơn, và chúng phản ứng với các mẫu phức tạp hơn là sự kết hợp của các mẫu cấp thấp hơn. Những quan sát này đã dẫn đến ý tưởng rằng các nơ-ron cấp cao hơn dựa trên đầu ra của các nơ-ron cấp thấp hơn lân cận (trong Hình 14-1, lưu ý rằng mỗi nơ-ron chỉ kết nối với các nơ-ron gần đó từ lớp trước). Kiến trúc mạnh mẽ này có khả năng phát hiện tất cả các loại mẫu phức tạp ở bất kỳ khu vực nào của trường thị giác.

Những nghiên cứu về vỏ não thị giác này đã truyền cảm hứng cho neocognitron, được giới thiệu vào năm 1980, sau đó dần phát triển thành cái mà chúng ta hiện gọi là mạng nơ-ron tích chập. Một cột mốc quan trọng là bài báo năm 1998 của Yann LeCun et al. đã giới thiệu kiến trúc LeNet-5 nổi tiếng, được các ngân hàng sử dụng rộng rãi để nhận dạng chữ số viết tay trên séc. Kiến trúc này có một số khối xây dựng mà bạn đã biết, chẳng hạn như các lớp kết nối đầy đủ và các hàm kích hoạt sigmoid, nhưng nó cũng giới thiệu hai khối xây dựng mới: các lớp tích chập và các lớp gộp. Bây giờ chúng ta hãy xem xét chúng.

Các lớp tích chập

Khối xây dựng quan trọng nhất của CNN là lớp tích chập: các nơ-ron trong lớp tích chập đầu tiên không được kết nối với mọi pixel trong hình ảnh đầu vào (như chúng đã ở trong các lớp được thảo luận trong các chương trước), mà chỉ kết nối với các pixel trong trường tiếp nhận của chúng (xem Hình 14-2).

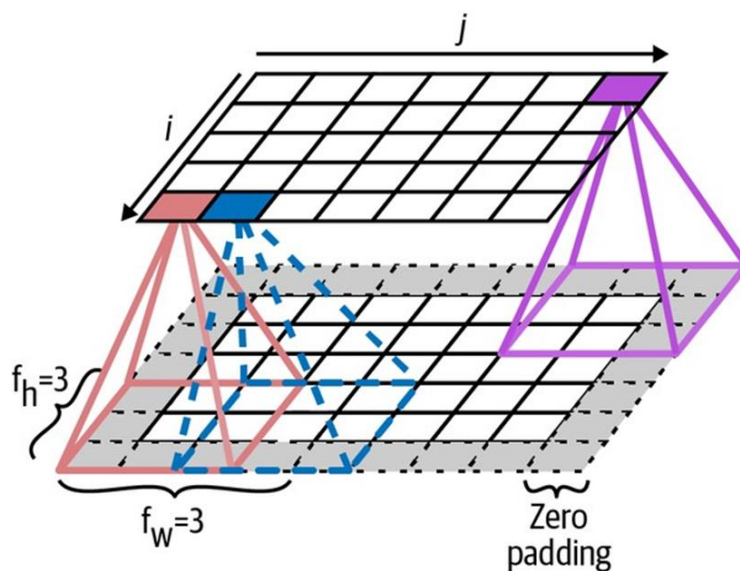
Đến lượt mình, mỗi nơ-ron trong lớp tích chập thứ hai chỉ được kết nối với các nơ-ron nằm trong một hình chữ nhật nhỏ ở lớp đầu tiên. Kiến trúc này cho phép mạng tập trung vào các đặc điểm cấp thấp nhỏ ở lớp ẩn đầu tiên, sau đó tập hợp chúng thành các đặc điểm cấp cao lớn hơn ở lớp ẩn tiếp theo, v.v. Cấu trúc phân cấp này phổ biến trong các hình ảnh thực tế, đây là một trong những lý do tại sao CNN hoạt động rất tốt cho nhận dạng hình ảnh.



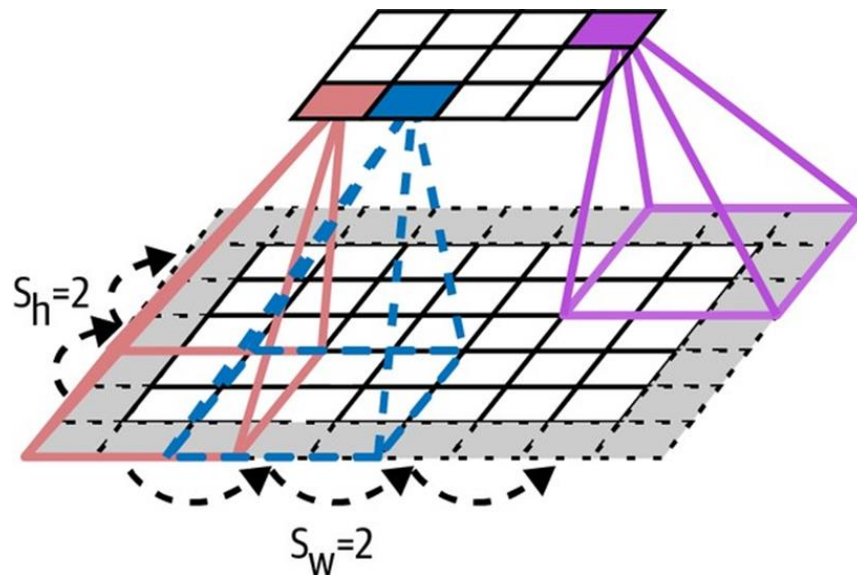
Hình 14-2. Các lớp CNN với trường tiếp nhận cục bộ hình chữ nhật.

Một nơ-ron nằm ở hàng i , cột j của một lớp nhất định được kết nối với đầu ra của các nơ-ron trong lớp trước nằm ở hàng i đến $i + f_h - 1$, cột j đến $j + f_w - 1$, trong đó f_h và f_w là chiều cao và chiều rộng của trường tiếp nhận (xem Hình 14-3). Để một lớp có cùng chiều cao và chiều rộng với lớp trước, việc thêm các số 0 xung quanh đầu vào là phổ biến, như thể hiện trong sơ đồ. Điều này được gọi là zero padding.

Cũng có thể kết nối một lớp đầu vào lớn với một lớp nhỏ hơn nhiều bằng cách giãn cách các trường tiếp nhận, như thể hiện trong Hình 14-4. Điều này làm giảm đáng kể độ phức tạp tính toán của mô hình. Kích thước bước ngang hoặc dọc từ trường tiếp nhận này sang trường tiếp nhận tiếp theo được gọi là stride. Trong sơ đồ, một lớp đầu vào 5×7 (cộng với zero padding) được kết nối với một lớp 3×4 , sử dụng các trường tiếp nhận 3×3 và một stride là 2 (trong ví dụ này, stride giống nhau ở cả hai hướng, nhưng không nhất thiết phải như vậy). Một nơ-ron nằm ở hàng i , cột j trong lớp trên được kết nối với đầu ra của các nơ-ron trong lớp trước nằm ở hàng $i \times s_h$ đến $i \times s_h + f_h - 1$, cột $j \times s_w$ đến $j \times s_w + f_w - 1$, trong đó s_h và s_w là các strides dọc và ngang.



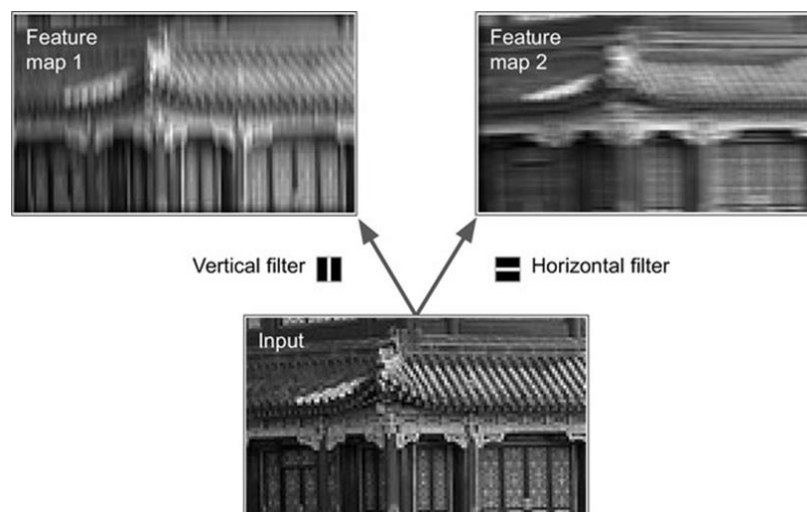
Hình 14-3. Các kết nối giữa các lớp và zero padding.



Hình 14-4. Giảm chiều bằng cách sử dụng stride là 2.

Bộ lọc

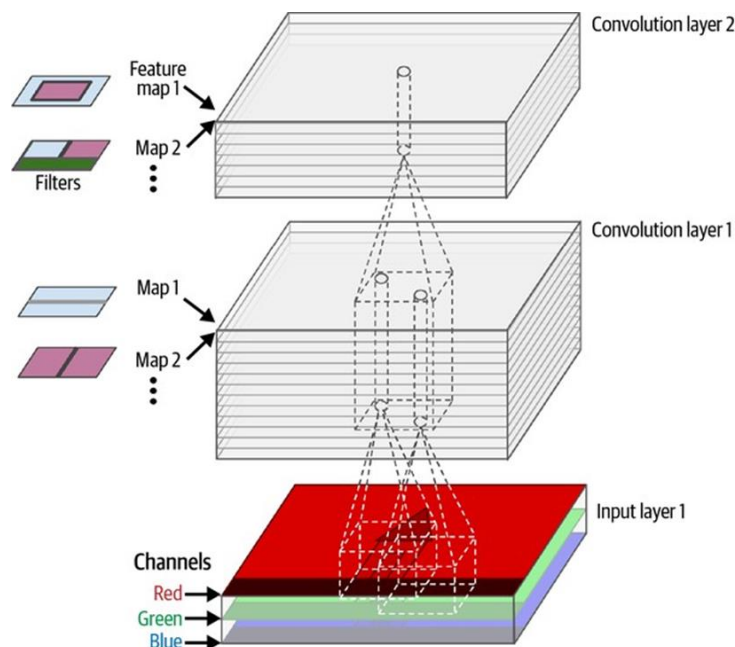
Các trọng số của một nơ-ron có thể được biểu diễn dưới dạng một hình ảnh nhỏ có kích thước bằng trường tiếp nhận. Ví dụ, Hình 14-5 cho thấy hai tập hợp trọng số có thể có, được gọi là bộ lọc (hoặc kernel tích chập, hoặc chỉ là kernel). Bộ lọc đầu tiên được biểu diễn dưới dạng một hình vuông màu đen với một đường thẳng đứng màu trắng ở giữa (đó là một ma trận 7×7 chứa toàn số 0 ngoại trừ cột trung tâm, chứa toàn số 1); các nơ-ron sử dụng các trọng số này sẽ bỏ qua mọi thứ trong trường tiếp nhận của chúng ngoại trừ đường thẳng đứng trung tâm (vì tất cả các đầu vào sẽ được nhân với 0, ngoại trừ những đầu vào nằm trên đường thẳng đứng trung tâm). Bộ lọc thứ hai là một hình vuông màu đen với một đường thẳng ngang màu trắng ở giữa. Các nơ-ron sử dụng các trọng số này sẽ bỏ qua mọi thứ trong trường tiếp nhận của chúng ngoại trừ đường thẳng ngang trung tâm.



Hình 14-5. Áp dụng hai bộ lọc khác nhau để có được hai bản đồ đặc trưng.

Bây giờ, nếu tất cả các nơ-ron trong một lớp sử dụng cùng một bộ lọc đường thẳng đứng (và cùng một thuật ngữ bias), và bạn cung cấp cho mạng hình ảnh đầu vào được hiển thị trong Hình 14-5 (hình ảnh dưới cùng), lớp đó sẽ xuất ra hình ảnh trên cùng bên trái. Lưu ý rằng các đường thẳng đứng màu trắng được tăng cường trong khi phần còn lại bị làm mờ. Tương tự, hình ảnh trên cùng bên phải là những gì bạn nhận được nếu tất cả các nơ-ron sử dụng cùng một bộ lọc đường thẳng ngang; lưu ý rằng các đường thẳng ngang màu trắng được tăng cường trong khi phần còn lại bị làm mờ. Do đó, một lớp đầy các nơ-ron sử dụng cùng một bộ lọc sẽ xuất ra một bản đồ đặc trưng, làm nổi bật các vùng trong một hình ảnh kích hoạt bộ lọc đó nhiều nhất. Nhưng đừng lo lắng, bạn sẽ không phải xác định các bộ lọc theo cách thủ công: thay vào đó, trong quá trình huấn luyện, lớp tích chập sẽ tự động học các bộ lọc hữu ích nhất cho tác vụ của nó, và các lớp phía trên sẽ học cách kết hợp chúng thành các mẫu phức tạp hơn.

Để thuận tiện, tôi đã trình bày đầu ra của mỗi lớp tích chập dưới dạng lớp 2D, nhưng trên thực tế, một lớp tích chập có nhiều bộ lọc (bạn quyết định bao nhiêu) và xuất ra một bản đồ đặc trưng cho mỗi bộ lọc, vì vậy nó được biểu diễn chính xác hơn ở dạng 3D (xem Hình 14-6). Nó có một nơ-ron trên mỗi pixel trong mỗi bản đồ đặc trưng, và tất cả các nơ-ron trong một bản đồ đặc trưng nhất định chia sẻ cùng các tham số (tức là cùng một kernel và thuật ngữ bias). Các nơ-ron trong các bản đồ đặc trưng khác nhau sử dụng các tham số khác nhau. Trường tiếp nhận của một nơ-ron giống như đã mô tả trước đó, nhưng nó mở rộng trên tất cả các bản đồ đặc trưng của lớp trước. Tóm lại, một lớp tích chập đồng thời áp dụng nhiều bộ lọc có thể huấn luyện cho đầu vào của nó, giúp nó có khả năng phát hiện nhiều đặc trưng ở bất cứ đâu trong đầu vào của nó.



Hình 14-6. Hai lớp tích chập với nhiều bộ lọc (kernels) mỗi lớp, xử lý một hình ảnh màu với ba kênh màu; mỗi lớp tích chập xuất ra một bản đồ đặc trưng cho mỗi bộ lọc.

Hình ảnh đầu vào cũng bao gồm nhiều lớp con: một lớp cho mỗi kênh màu. Như đã đề cập trong Chương 9, thường có ba kênh: đỏ, xanh lá cây và xanh lam (RGB). Hình ảnh thang độ xám chỉ có một kênh, nhưng một số hình ảnh có thể có nhiều kênh hơn—ví dụ, hình ảnh vệ tinh chụp các tần số ánh sáng bổ sung (chẳng hạn như hồng ngoại).

Cụ thể, một nơ-ron nằm ở hàng i , cột j của bản đồ đặc trưng k trong một lớp tích chập l nhất định được kết nối với đầu ra của các nơ-ron nhất định trong lớp trước $l - 1$, nằm ở hàng $i \times s_h$ đến $i \times s_h + f_h - 1$ và cột $j \times s_w$ đến $j \times s_w + f_w - 1$, trên tất cả các bản đồ đặc trưng (trong lớp $l - 1$). Lưu ý rằng, trong một lớp, tất cả các nơ-ron nằm ở cùng một hàng i và cột j nhưng trong các bản đồ đặc trưng khác nhau được kết nối với đầu ra của chính xác cùng các nơ-ron trong lớp trước.

Phương trình 14-1 tóm tắt các giải thích trên trong một phương trình toán học lớn: nó cho thấy cách tính đầu ra của một nơ-ron nhất định trong một lớp tích chập. Nó hơi khó nhìn do tất cả các chỉ số khác nhau, nhưng tất cả những gì nó làm là tính tổng trọng số của tất cả các đầu vào, cộng với thuật ngữ bias.

Phương trình 14-1. Tính toán đầu ra của một nơ-ron trong một lớp tích chập

$$z_{i,j,k} = b_k + \sum_{u=0}^{f_h-1} \sum_{v=0}^{f_w-1} \sum_{k'=0}^{f_n-1} x_{i',j',k'} \times w_{u,v,k',k} \quad \text{with} \quad \begin{cases} i' = i \times s_h + u \\ j' = j \times s_w + v \end{cases}$$

Trong phương trình này:

- $z_{i,j,k}$ là đầu ra của nơ-ron nằm ở hàng i , cột j trong bản đồ đặc trưng k của lớp tích chập (lớp l).
- Như đã giải thích trước đó, s_h và s_w là các stride dọc và ngang, f_h và f_w là chiều cao và chiều rộng của trường tiếp nhận, và f_n là số lượng bản đồ đặc trưng trong lớp trước (lớp $l - 1$).
- $x_{i',j',k'}$ là đầu ra của nơ-ron nằm ở lớp $l - 1$, hàng i' , cột j' , bản đồ đặc trưng k' (hoặc kênh k' nếu lớp trước là lớp đầu vào).
- b_k là thuật ngữ bias cho bản đồ đặc trưng k (trong lớp l). Bạn có thể nghĩ nó như một nút điều chỉnh độ sáng tổng thể của bản đồ đặc trưng k .
- $w_{u,v,k',k}$ là trọng số kết nối giữa bất kỳ nơ-ron nào trong bản đồ đặc trưng k của lớp l và đầu vào của nó nằm ở hàng u , cột v (so với trường tiếp nhận của nơ-ron), và bản đồ đặc trưng k' .

Hãy xem cách tạo và sử dụng một lớp tích chập bằng Keras.

Triển khai các lớp tích chập với Keras

Đầu tiên, hãy tải và tiền xử lý một vài hình ảnh mẫu, sử dụng hàm `load_sample_image()` của Scikit-Learn và các lớp `CenterCrop` và `Rescaling` của Keras (tất cả đều đã được giới thiệu trong Chương 13):

```
images = load_sample_images()["images"]
images = tf.keras.layers.CenterCrop(height=70, width=120)(images)
images = tf.keras.layers.Rescaling(scale=1 / 255)(images)
```

Hãy xem hình dạng của tensor images:

```
>>> images.shape
TensorShape([2, 70, 120, 3])
```

Ồi, đó là một tensor 4D; chúng ta chưa từng thấy điều này trước đây! Tất cả các chiều này có nghĩa là gì? Vâng, có hai hình ảnh mẫu, điều này giải thích chiều đầu tiên. Sau đó, mỗi hình ảnh là 70×120 , vì đó là kích thước chúng ta đã chỉ định khi tạo lớp CenterCrop (các hình ảnh gốc là 427×640). Điều này giải thích chiều thứ hai và thứ ba. Và cuối cùng, mỗi pixel chứa một giá trị trên mỗi kênh màu, và có ba kênh—đỏ, xanh lá cây và xanh lam—điều này giải thích chiều cuối cùng.

Bây giờ hãy tạo một lớp tích chập 2D và đưa các hình ảnh này vào để xem kết quả. Đối với điều này, Keras cung cấp một lớp Convolution2D, bí danh Conv2D. Bên dưới, lớp này dựa vào thao tác `tf.nn.conv2d()` của TensorFlow. Hãy tạo một lớp tích chập với 32 bộ lọc, mỗi bộ lọc có kích thước 7×7 (sử dụng `kernel_size=7`, tương đương với việc sử dụng `kernel_size=(7, 7)`), và áp dụng lớp này cho lô nhỏ gồm hai hình ảnh của chúng ta:

```
conv_layer = tf.keras.layers.Conv2D(filters=32, kernel_size=7)
fmaps = conv_layer(images)
```

Bây giờ hãy xem hình dạng của đầu ra:

```
>>> fmaps.shape
TensorShape([2, 64, 114, 32])
```

Hình dạng đầu ra tương tự như hình dạng đầu vào, với hai khác biệt chính. Đầu tiên, có 32 kênh thay vì 3. Điều này là do chúng ta đã đặt `filters=32`, vì vậy chúng ta nhận được 32 bản đồ đặc trưng đầu ra: thay vì cường độ của màu đỏ, xanh lá cây và xanh lam tại mỗi vị trí, bây giờ chúng ta có cường độ của mỗi đặc trưng tại mỗi vị trí.

Thứ hai, chiều cao và chiều rộng đều giảm đi 6 pixel. Điều này là do lớp Conv2D không sử dụng bất kỳ zero-padding nào theo mặc định, điều đó có nghĩa là chúng ta mất một vài pixel ở các cạnh của bản đồ đặc trưng đầu ra, tùy thuộc vào kích thước của các bộ lọc. Trong trường hợp này, vì kích thước kernel là 7, chúng ta mất 6 pixel theo chiều ngang và 6 pixel theo chiều dọc (tức là 3 pixel ở mỗi bên).

Nếu thay vào đó chúng ta đặt `padding="same"`, thì đầu vào sẽ được đệm bằng đủ số 0 ở tất cả các phía để đảm bảo rằng các bản đồ đặc trưng đầu ra có cùng kích thước với đầu vào (do đó có tên của tùy chọn này):

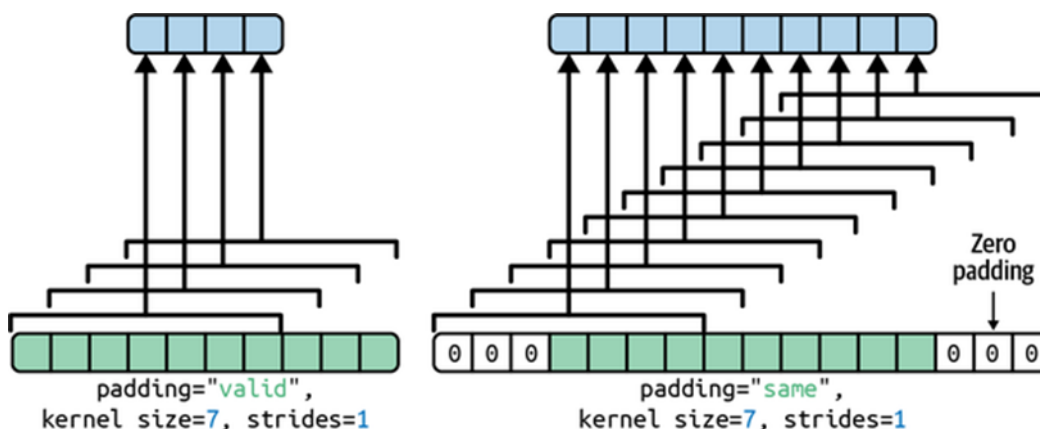
```
>>> conv_layer = tf.keras.layers.Conv2D(filters=32, kernel_size=7,
...                                     padding="same")
...
...
>>> fmaps = conv_layer(images)
```



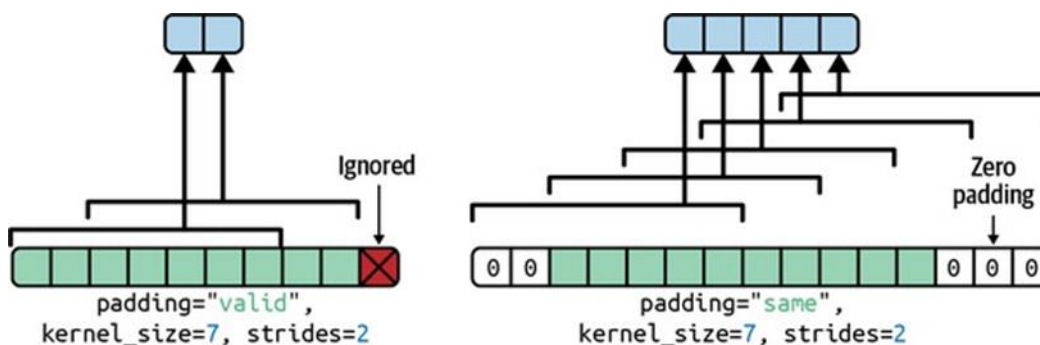
```
>>> fmaps.shape
TensorShape([2, 70, 120, 32])
```

Hai tùy chọn padding này được minh họa trong Hình 14-7. Để đơn giản, chỉ có chiều ngang được hiển thị ở đây, nhưng tất nhiên logic tương tự cũng áp dụng cho chiều dọc.

Nếu stride lớn hơn 1 (theo bất kỳ hướng nào), thì kích thước đầu ra sẽ không bằng kích thước đầu vào, ngay cả khi padding="same". Ví dụ, nếu bạn đặt strides=2 (hoặc tương đương strides=(2, 2)), thì các bản đồ đặc trưng đầu ra sẽ là 35 × 60: giảm một nửa cả theo chiều dọc và chiều ngang. Hình 14-8 cho thấy điều gì xảy ra khi strides=2, với cả hai tùy chọn padding.



Hình 14-7. Hai tùy chọn padding, khi strides=1.



Hình 14-8. Với strides lớn hơn 1, đầu ra nhỏ hơn nhiều ngay cả khi sử dụng padding “same” (và padding “valid” có thể bỏ qua một số đầu vào).

Nếu bạn tò mò, đây là cách tính kích thước đầu ra:

- Với padding="valid", nếu chiều rộng của đầu vào là i_h , thì chiều rộng đầu ra bằng $(i_h - f_h + s_h) / s_h$, làm tròn xuống. Nhắc lại rằng f_h là chiều rộng kernel, và s_h là stride ngang. Bất kỳ phần dư nào trong phép chia đều tương ứng với các cột bị bỏ qua ở phía bên phải của hình ảnh đầu vào. Logic tương tự có thể được sử dụng để tính chiều cao đầu ra và bất kỳ hàng bị bỏ qua nào ở cuối hình ảnh.

- Với `padding="same"`, chiều rộng đầu ra bằng ih / sh , làm tròn lên. Để điều này có thể thực hiện được, số cột 0 thích hợp được đệm vào bên trái và bên phải của hình ảnh đầu vào (số lượng bằng nhau nếu có thể, hoặc chỉ nhiều hơn một cột ở phía bên phải). Giả sử chiều rộng đầu ra là ow , thì số cột 0 được đệm là $(ow - 1) \times sh + fh - ih$. Một lần nữa, logic tương tự có thể được sử dụng để tính chiều cao đầu ra và số hàng được đệm.

Bây giờ hãy xem xét các trọng số của lớp (được ký hiệu là w_u, v, k', k và b_k trong Phương trình 14-1). Giống như một lớp Dense, một lớp Conv2D chứa tất cả các trọng số của lớp, bao gồm các kernel và bias. Các kernel được khởi tạo ngẫu nhiên, trong khi các bias được khởi tạo bằng 0. Các trọng số này có thể truy cập được dưới dạng các biến TF thông qua thuộc tính `weights`, hoặc dưới dạng các mảng NumPy thông qua phương thức `get_weights()`:

```
>>> kernels, biases = conv_layer.get_weights()
>>> kernels.shape
(7, 7, 3, 32)
>>> biases.shape
(32,)
```

Mảng `kernels` là 4D, và hình dạng của nó là `[kernel_height, kernel_width, input_channels, output_channels]`. Mảng `biases` là 1D, với hình dạng `[output_channels]`. Số lượng kênh đầu ra bằng số lượng bản đồ đặc trưng đầu ra, cũng bằng số lượng bộ lọc.

Quan trọng nhất, lưu ý rằng chiều cao và chiều rộng của hình ảnh đầu vào không xuất hiện trong hình dạng của kernel: điều này là do tất cả các nơ-ron trong các bản đồ đặc trưng đầu ra chia sẻ cùng một trọng số, như đã giải thích trước đó. Điều này có nghĩa là bạn có thể đưa hình ảnh có bất kỳ kích thước nào vào lớp này, miễn là chúng ít nhất lớn bằng các kernel, và nếu chúng có đúng số lượng kênh (ba trong trường hợp này).

Cuối cùng, bạn thường muốn chỉ định một hàm kích hoạt (chẳng hạn như ReLU) khi tạo một lớp Conv2D, và cũng chỉ định trình khởi tạo kernel tương ứng (chẳng hạn như He initialization). Điều này cũng vì lý do tương tự như đối với các lớp Dense: một lớp tích chập thực hiện một phép toán tuyến tính, vì vậy nếu bạn xếp chồng nhiều lớp tích chập mà không có bất kỳ hàm kích hoạt nào, tất cả chúng sẽ tương đương với một lớp tích chập duy nhất, và chúng sẽ không thể học được bất cứ điều gì thực sự phức tạp.

Như bạn có thể thấy, các lớp tích chập có khá nhiều siêu tham số: `filters`, `kernel_size`, `padding`, `strides`, `activation`, `kernel_initializer`, v.v. Như mọi khi, bạn có thể sử dụng kiểm định chéo để tìm ra các giá trị siêu tham số phù hợp, nhưng điều này rất tốn thời gian. Chúng ta sẽ thảo luận về các kiến trúc CNN phổ biến sau này trong chương này, để cung cấp cho bạn một số ý tưởng về giá trị siêu tham số nào hoạt động tốt nhất trong thực tế.

Yêu cầu về bộ nhớ

Một thách thức khác với CNN là các lớp tích chập yêu cầu lượng RAM khổng lồ. Điều này đặc biệt đúng trong quá trình huấn luyện, vì lượt ngược của lan truyền ngược yêu cầu tất cả các giá trị trung gian được tính toán trong lượt thuận.

Ví dụ, hãy xem xét một lớp tích chập với 200 bộ lọc 5×5 , với stride 1 và padding “same”. Nếu đầu vào là hình ảnh RGB 150×100 (ba kênh), thì số lượng tham số là $(5 \times 5 \times 3 + 1) \times 200 = 15.200$ (cái + 1 tương ứng với các thuật ngữ bias), khá nhỏ so với một lớp kết nối đầy đủ. Tuy nhiên, mỗi trong số 200 bản đồ đặc trưng chứa 150×100 nơ-ron, và mỗi nơ-ron này cần tính tổng trọng số của $5 \times 5 \times 3 = 75$ đầu vào của nó: tổng cộng là 225 triệu phép nhân dấu phẩy động. Không tệ bằng một lớp kết nối đầy đủ, nhưng vẫn khá tốn kém về mặt tính toán.

Hơn nữa, nếu các bản đồ đặc trưng được biểu diễn bằng số dấu phẩy động 32 bit, thì đầu ra của lớp tích chập sẽ chiếm $200 \times 150 \times 100 \times 32 = 96$ triệu bit (12 MB) RAM. Và đó chỉ là cho một trường hợp—nếu một lô huấn luyện chứa 100 trường hợp, thì lớp này sẽ sử dụng tới 1,2 GB RAM!

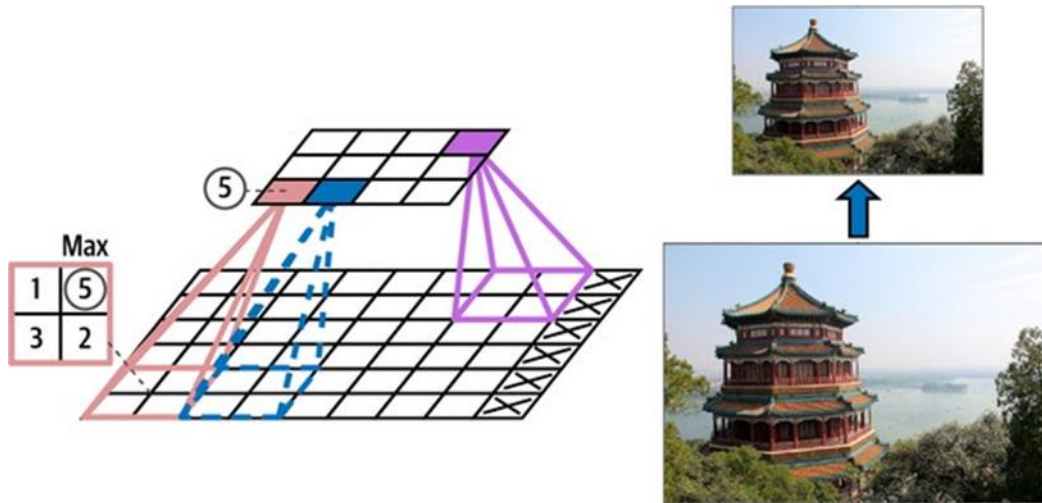
Trong quá trình suy luận (tức là khi đưa ra dự đoán cho một trường hợp mới), RAM bị chiếm bởi một lớp có thể được giải phóng ngay sau khi lớp tiếp theo đã được tính toán, vì vậy bạn chỉ cần lượng RAM cần thiết cho hai lớp liên tiếp. Nhưng trong quá trình huấn luyện, mọi thứ được tính toán trong lượt thuận cần phải được bảo toàn cho lượt ngược, vì vậy lượng RAM cần thiết là (ít nhất) tổng lượng RAM cần thiết cho tất cả các lớp.

Bây giờ chúng ta hãy xem khối xây dựng phổ biến thứ hai của CNN: lớp gộp (pooling layer).

Các lớp gộp

Khi bạn hiểu cách các lớp tích chập hoạt động, các lớp gộp khá dễ nắm bắt. Mục tiêu của chúng là lấy mẫu con (tức là thu nhỏ) hình ảnh đầu vào để giảm tải tính toán, sử dụng bộ nhớ và số lượng tham số (do đó hạn chế rủi ro quá khớp).

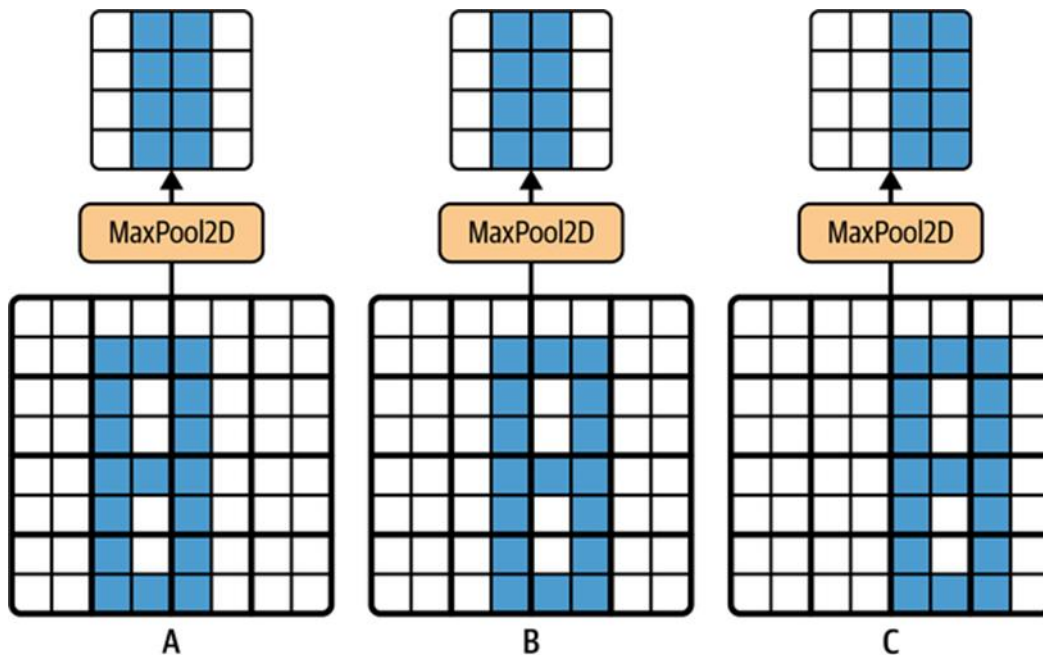
Giống như trong các lớp tích chập, mỗi nơ-ron trong một lớp gộp được kết nối với đầu ra của một số lượng hạn chế các nơ-ron trong lớp trước, nằm trong một trường tiếp nhận hình chữ nhật nhỏ. Bạn phải xác định kích thước của nó, stride và loại padding, giống như trước đây. Tuy nhiên, một nơ-ron gộp không có trọng số; tất cả những gì nó làm là tổng hợp các đầu vào bằng cách sử dụng một hàm tổng hợp như max hoặc mean. Hình 14-9 cho thấy một lớp gộp max (max pooling layer), là loại lớp gộp phổ biến nhất. Trong ví dụ này, chúng ta sử dụng một kernel gộp 2×2 , với stride là 2 và không padding. Chỉ giá trị đầu vào max trong mỗi trường tiếp nhận mới được đưa đến lớp tiếp theo, trong khi các đầu vào khác bị loại bỏ. Ví dụ, trong trường tiếp nhận phía dưới bên trái trong Hình 14-9, các giá trị đầu vào là 1, 5, 3, 2, vì vậy chỉ giá trị max là 5 được truyền đến lớp tiếp theo. Do stride là 2, hình ảnh đầu ra có chiều cao bằng một nửa và chiều rộng bằng một nửa hình ảnh đầu vào (làm tròn xuống vì chúng ta không sử dụng padding).



Hình 14-9. Lớp gộp max (kernel gộp 2×2 , stride 2, không padding).

Ngoài việc giảm tính toán, sử dụng bộ nhớ và số lượng tham số, một lớp gộp max còn giới thiệu một mức độ bất biến đối với các phép dịch chuyển nhỏ, như thể hiện trong Hình 14-10. Ở đây chúng ta giả định rằng các pixel sáng có giá trị thấp hơn các pixel tối, và chúng ta xem xét ba hình ảnh (A, B, C) đi qua một lớp gộp max với kernel 2×2 và stride 2. Hình ảnh B và C giống như hình ảnh A, nhưng dịch chuyển một và hai pixel sang phải. Như bạn có thể thấy, đầu ra của lớp gộp max cho hình ảnh A và B là giống hệt nhau. Đây là ý nghĩa của bất biến dịch chuyển. Đối với hình ảnh C, đầu ra khác: nó được dịch chuyển một pixel sang phải (nhưng vẫn có 50% bất biến). Bằng cách chèn một lớp gộp max sau mỗi vài lớp trong một CNN, có thể đạt được một mức độ bất biến dịch chuyển ở quy mô lớn hơn. Hơn nữa, gộp max cung cấp một lượng nhỏ bất biến xoay và một chút bất biến tỷ lệ. Bất biến như vậy (ngay cả khi nó bị hạn chế) có thể hữu ích trong các trường hợp mà dự đoán không nên phụ thuộc vào các chi tiết này, chẳng hạn như trong các tác vụ phân loại.

Tuy nhiên, gộp max cũng có một số nhược điểm. Nó rõ ràng là rất phá hủy: ngay cả với một kernel 2×2 nhỏ và stride 2, đầu ra sẽ nhỏ hơn hai lần theo cả hai hướng (vì vậy diện tích của nó sẽ nhỏ hơn bốn lần), đơn giản là loại bỏ 75% giá trị đầu vào. Và trong một số ứng dụng, bất biến không mong muốn. Lấy phân đoạn ngữ nghĩa (nhiệm vụ phân loại từng pixel trong một hình ảnh theo đối tượng mà pixel đó thuộc về, chúng ta sẽ khám phá sau trong chương này): rõ ràng, nếu hình ảnh đầu vào được dịch chuyển một pixel sang phải, đầu ra cũng phải được dịch chuyển một pixel sang phải. Mục tiêu trong trường hợp này là đồng biến, không phải bất biến: một thay đổi nhỏ đối với đầu vào sẽ dẫn đến một thay đổi nhỏ tương ứng trong đầu ra.



Hình 14-10. Bất biến đối với các phép dịch chuyển nhỏ.

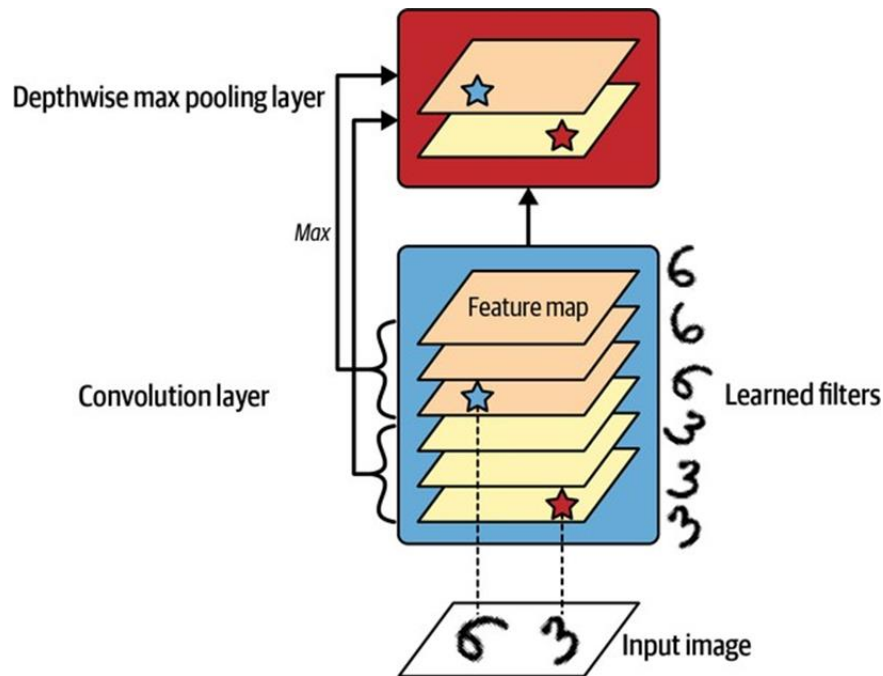
Triển khai các lớp gộp với Keras

Mã sau đây tạo một lớp MaxPooling2D, bí danh MaxPool2D, sử dụng một kernel 2×2 . Các stride mặc định là kích thước kernel, vì vậy lớp này sử dụng stride là 2 (ngang và dọc). Theo mặc định, nó sử dụng padding “valid” (tức là không padding gì cả):

```
max_pool = tf.keras.layers.MaxPool2D(pool_size=2)
```

Để tạo một lớp gộp trung bình, chỉ cần sử dụng AveragePooling2D, bí danh AvgPool2D, thay vì MaxPool2D. Như bạn có thể mong đợi, nó hoạt động chính xác như một lớp gộp max, ngoại trừ nó tính toán giá trị trung bình chứ không phải giá trị max. Các lớp gộp trung bình từng rất phổ biến, nhưng bây giờ mọi người chủ yếu sử dụng các lớp gộp max, vì chúng thường hoạt động tốt hơn. Điều này có vẻ đáng ngạc nhiên, vì tính toán giá trị trung bình thường mất ít thông tin hơn so với tính toán giá trị max. Nhưng mặt khác, gộp max chỉ bảo tồn các đặc trưng mạnh nhất, loại bỏ tất cả các đặc trưng vô nghĩa, vì vậy các lớp tiếp theo có một tín hiệu sạch hơn để làm việc. Hơn nữa, gộp max cung cấp bất biến dịch chuyển mạnh hơn so với gộp trung bình, và nó yêu cầu ít tính toán hơn một chút.

Lưu ý rằng gộp max và gộp trung bình có thể được thực hiện dọc theo chiều sâu thay vì các chiều không gian, mặc dù nó không phổ biến bằng. Điều này có thể cho phép CNN học cách bất biến đối với các đặc trưng khác nhau. Ví dụ, nó có thể học nhiều bộ lọc, mỗi bộ lọc phát hiện một phép xoay khác nhau của cùng một mẫu (chẳng hạn như chữ số viết tay; xem Hình 14-11), và lớp gộp max theo chiều sâu sẽ đảm bảo rằng đầu ra giống nhau bất kể phép xoay. CNN cũng có thể học cách bất biến đối với bất cứ điều gì: độ dày, độ sáng, độ xiên, màu sắc, v.v.



Hình 14-11. Gộp max theo chiều sâu có thể giúp CNN học cách bất biến (trong trường hợp này là xoay).

Keras không bao gồm một lớp gộp max theo chiều sâu, nhưng không quá khó để triển khai một lớp tùy chỉnh cho việc đó:

```
class DepthPool(tf.keras.layers.Layer):
    def __init__(self, pool_size=2, **kwargs):
        super().__init__(**kwargs)
        self.pool_size = pool_size

    def call(self, inputs):
        shape = tf.shape(inputs) # shape[-1] is the number of channels
        groups = shape[-1] // self.pool_size # number of channel groups
        new_shape = tf.concat([shape[:-1], [groups, self.pool_size]], axis=0)
        return tf.reduce_max(tf.reshape(inputs, new_shape), axis=-1)
```

Lớp này định hình lại đầu vào của nó để chia các kênh thành các nhóm có kích thước mong muốn (`pool_size`), sau đó nó sử dụng `tf.reduce_max()` để tính toán giá trị max của mỗi nhóm. Việc triển khai này giả định rằng `stride` bằng kích thước `pool`, đây thường là điều bạn muốn. Ngoài ra, bạn có thể sử dụng thao tác `tf.nn.max_pool()` của TensorFlow và gói trong một lớp `Lambda` để sử dụng nó bên trong một mô hình Keras, nhưng đáng buồn là thao tác này không triển khai gộp theo chiều sâu cho GPU, chỉ cho CPU.

Một loại lớp gộp cuối cùng mà bạn thường thấy trong các kiến trúc hiện đại là lớp gộp trung bình toàn cục (`global average pooling layer`). Nó hoạt động rất khác: tất cả những gì nó làm là tính toán giá trị trung bình của mỗi toàn bộ bản đồ đặc trưng (nó giống như một lớp gộp trung bình sử dụng kernel gộp có cùng kích thước không gian với đầu vào). Điều này có nghĩa là nó chỉ xuất ra một số duy nhất cho mỗi bản đồ đặc trưng và mỗi trường hợp.

Mặc dù điều này rõ ràng là cực kỳ phá hủy (phần lớn thông tin trong bản đồ đặc trưng bị mất), nhưng nó có thể hữu ích ngay trước lớp đầu ra, như bạn sẽ thấy sau trong chương này. Để tạo một lớp như vậy, chỉ cần sử dụng lớp `GlobalAveragePooling2D`, bí danh `GlobalAvgPool2D`:

```
global_avg_pool = tf.keras.layers.GlobalAvgPool2D()
```

Nó tương đương với lớp `Lambda` sau đây, lớp này tính toán giá trị trung bình trên các chiều không gian (chiều cao và chiều rộng):

```
global_avg_pool = tf.keras.layers.Lambda(  
    lambda X: tf.reduce_mean(X, axis=[1, 2])  
)
```

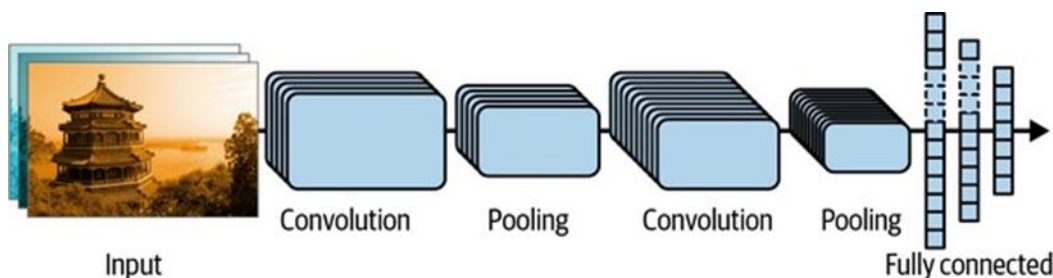
Ví dụ, nếu chúng ta áp dụng lớp này cho hình ảnh đầu vào, chúng ta sẽ nhận được cường độ trung bình của màu đỏ, xanh lá cây và xanh lam cho mỗi hình ảnh:

```
>>> global_avg_pool(images)  
<tf.Tensor: shape=(2, 3), dtype=float32, numpy=  
array([[0.64338624, 0.5971759 , 0.5824972 ],  
       [0.76306933, 0.26011038, 0.10849128]], dtype=float32)>
```

Bây giờ bạn đã biết tất cả các khối xây dựng để tạo mạng nơ-ron tích chập. Hãy xem cách lắp ráp chúng.

Kiến trúc CNN

Các kiến trúc CNN điển hình xếp chồng một vài lớp tích chập (mỗi lớp thường theo sau bởi một lớp `ReLU`), sau đó là một lớp gộp, sau đó lại vài lớp tích chập khác (+`ReLU`), sau đó lại một lớp gộp, v.v. Hình ảnh ngày càng nhỏ đi khi nó đi qua mạng, nhưng nó cũng thường trở nên sâu hơn (tức là có nhiều bản đồ đặc trưng hơn), nhờ các lớp tích chập (xem Hình 14-12). Ở trên cùng của chồng, một mạng nơ-ron truyền thẳng thông thường được thêm vào, bao gồm một vài lớp kết nối đầy đủ (+`ReLU`), và lớp cuối cùng xuất ra dự đoán (ví dụ: một lớp softmax xuất ra xác suất lớp ước tính).



Hình 14-12. Kiến trúc CNN điển hình.

Dưới đây là cách bạn có thể triển khai một CNN cơ bản để giải quyết tập dữ liệu Fashion MNIST (được giới thiệu trong Chương 10):

```

from functools import partial

DefaultConv2D = partial(tf.keras.layers.Conv2D, kernel_size=3,
padding="same",
                        activation="relu", kernel_initializer="he_normal")

model = tf.keras.Sequential([
    DefaultConv2D(filters=64, kernel_size=7, input_shape=[28, 28, 1]),
    tf.keras.layers.MaxPool2D(),
    DefaultConv2D(filters=128),
    DefaultConv2D(filters=128),
    tf.keras.layers.MaxPool2D(),
    DefaultConv2D(filters=256),
    DefaultConv2D(filters=256),
    tf.keras.layers.MaxPool2D(),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(units=128, activation="relu",
                        kernel_initializer="he_normal"),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(units=64, activation="relu",
                        kernel_initializer="he_normal"),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(units=10, activation="softmax")
])

```

Hãy cùng xem qua đoạn mã này:

- Chúng ta sử dụng hàm `functools.partial()` (được giới thiệu trong Chương 11) để định nghĩa `DefaultConv2D`, hoạt động giống như `Conv2D` nhưng với các đối số mặc định khác: kích thước kernel nhỏ là 3, padding “same”, hàm kích hoạt ReLU và trình khởi tạo He tương ứng.
- Tiếp theo, chúng ta tạo mô hình `Sequential`. Lớp đầu tiên của nó là `DefaultConv2D` với 64 bộ lọc khá lớn (7×7). Nó sử dụng stride mặc định là 1 vì hình ảnh đầu vào không quá lớn. Nó cũng đặt `input_shape=[28, 28, 1]`, vì hình ảnh có kích thước 28×28 pixel, với một kênh màu duy nhất (tức là thang độ xám). Khi bạn tải tập dữ liệu Fashion MNIST, hãy đảm bảo mỗi hình ảnh có hình dạng này: bạn có thể cần sử dụng `np.reshape()` hoặc `np.expanddims()` để thêm chiều kênh. Hoặc, bạn có thể sử dụng lớp `Reshape` làm lớp đầu tiên trong mô hình.
- Sau đó, chúng ta thêm một lớp gộp max sử dụng kích thước pool mặc định là 2, vì vậy nó chia mỗi chiều không gian cho hệ số 2.
- Sau đó, chúng ta lặp lại cấu trúc tương tự hai lần: hai lớp tích chập theo sau bởi một lớp gộp max. Đối với hình ảnh lớn hơn, chúng ta có thể lặp lại cấu trúc này nhiều lần hơn. Số lần lặp lại là một siêu tham số mà bạn có thể điều chỉnh.
- Lưu ý rằng số lượng bộ lọc tăng gấp đôi khi chúng ta đi lên CNN về phía lớp đầu ra (ban đầu là 64, sau đó là 128, sau đó là 256): điều này hợp lý vì số lượng đặc trưng cấp thấp thường khá thấp (ví dụ: các vòng tròn nhỏ, đường ngang), nhưng có nhiều

cách khác nhau để kết hợp chúng thành các đặc trưng cấp cao hơn. Một thực hành phổ biến là tăng gấp đôi số lượng bộ lọc sau mỗi lớp gộp: vì một lớp gộp chia mỗi chiều không gian cho hệ số 2, chúng ta có thể tăng gấp đôi số lượng bản đồ đặc trưng trong lớp tiếp theo mà không sợ làm bùng nổ số lượng tham số, mức sử dụng bộ nhớ hoặc tải tính toán.

- Tiếp theo là mạng kết nối đầy đủ, bao gồm hai lớp ẩn dày đặc và một lớp đầu ra dày đặc. Vì đây là một tác vụ phân loại với 10 lớp, lớp đầu ra có 10 đơn vị và nó sử dụng hàm kích hoạt softmax. Lưu ý rằng chúng ta phải làm phẳng đầu vào ngay trước lớp dày đặc đầu tiên, vì nó mong đợi một mảng tính năng 1D cho mỗi phiên bản. Chúng ta cũng thêm hai lớp dropout, với tỷ lệ dropout 50% mỗi lớp, để giảm overfitting.

Nếu bạn biên dịch mô hình này bằng cách sử dụng hàm mất mát “sparse_categorical_crossentropy” và bạn huấn luyện mô hình trên tập huấn luyện Fashion MNIST, nó sẽ đạt được độ chính xác trên 92% trên tập kiểm tra. Nó không phải là công nghệ tiên tiến nhất, nhưng nó khá tốt và rõ ràng là tốt hơn nhiều so với những gì chúng ta đã đạt được với các mạng dày đặc trong Chương 10.

Trong những năm qua, các biến thể của kiến trúc cơ bản này đã được phát triển, dẫn đến những tiến bộ đáng kinh ngạc trong lĩnh vực này. Một thước đo tốt về sự tiến bộ này là tỷ lệ lỗi trong các cuộc thi như thử thách ILSVRC ImageNet. Trong cuộc thi này, tỷ lệ lỗi top-5 cho phân loại hình ảnh—tức là số lượng hình ảnh thử nghiệm mà hệ thống không bao gồm câu trả lời đúng trong năm dự đoán hàng đầu của nó—đã giảm từ hơn 26% xuống dưới 2,3% chỉ trong sáu năm. Các hình ảnh khá lớn (ví dụ: cao 256 pixel) và có 1.000 lớp, một số trong số đó thực sự rất khó phân biệt (thử phân biệt 120 giống chó). Nhìn vào sự tiến hóa của các mục chiến thắng là một cách tốt để hiểu cách CNN hoạt động và cách nghiên cứu trong học sâu tiến bộ.

Chúng ta sẽ xem xét kiến trúc LeNet-5 cổ điển (1998) trước, sau đó là một số người chiến thắng thử thách ILSVRC: AlexNet (2012), GoogLeNet (2014), ResNet (2015) và SENet (2017). Ngoài ra, chúng ta cũng sẽ xem xét một vài kiến trúc khác, bao gồm Xception, ResNeXt, DenseNet, MobileNet, CSPNet và EfficientNet.

LeNet-5

Kiến trúc LeNet-5 có lẽ là kiến trúc CNN được biết đến rộng rãi nhất. Như đã đề cập trước đó, nó được tạo ra bởi Yann LeCun vào năm 1998 và đã được sử dụng rộng rãi để nhận dạng chữ số viết tay (MNIST). Nó bao gồm các lớp được thể hiện trong Bảng 14-1.

Bảng 14-1. Kiến trúc LeNet-5

Lớp	Loại	Bản đồ	Kích thước	Kích thước Kernel	Stride
Out	Fully connected	–	10	–	–
F6	Fully connected	–	84	–	–
C5	Convolution	120	1 × 1	5 × 5	1
S4	Avg pooling	16	5 × 5	2 × 2	2

Lớp	Loại	Bản đồ	Kích thước	Kích thước Kernel	Stride
C3	Convolution	16	10×10	5×5	1
S2	Avg pooling	6	14×14	2×2	2
C1	Convolution	6	28×28	5×5	1
In	Input	1	32×32	–	–

Như bạn có thể thấy, điều này trông khá giống với mô hình Fashion MNIST của chúng ta: một chồng các lớp tích chập và lớp gộp, sau đó là một mạng dày đặc. Có lẽ sự khác biệt chính với các CNN phân loại hiện đại hơn là các hàm kích hoạt: ngày nay, chúng ta sẽ sử dụng ReLU thay vì tanh và softmax thay vì RBF. Có một số khác biệt nhỏ khác không thực sự quan trọng lắm, nhưng trong trường hợp bạn quan tâm, chúng được liệt kê trong sổ tay của chương này tại <https://homl.info/colab3>. Trang web của Yann LeCun cũng có các bản demo tuyệt vời về LeNet-5 phân loại chữ số.

AlexNet

Kiến trúc AlexNet CNN đã giành chiến thắng thử thách ILSVRC 2012 với tỷ lệ lớn: nó đạt tỷ lệ lỗi top-5 là 17%, trong khi đối thủ tốt thứ hai chỉ đạt 26%! AlexNet được phát triển bởi Alex Krizhevsky (do đó có tên), Ilya Sutskever và Geoffrey Hinton. Nó tương tự như LeNet-5, chỉ lớn hơn và sâu hơn nhiều, và nó là kiến trúc đầu tiên xếp chồng trực tiếp các lớp tích chập lên nhau, thay vì xếp chồng một lớp gộp lên trên mỗi lớp tích chập. Bảng 14-2 trình bày kiến trúc này.

Bảng 14-2. Kiến trúc AlexNet

Lớp	Loại	Bản đồ	Kích thước	Kích thước Kernel	Stride
Out	Fully connected	–	1,000	–	–
F10	Fully connected	–	4,096	–	–
F9	Fully connected	–	4,096	–	–
S8	Max pooling	256	6×6	3×3	2
C7	Convolution	256	13×13	3×3	1
C6	Convolution	384	13×13	3×3	1
C5	Convolution	384	13×13	3×3	1
S4	Max pooling	256	13×13	3×3	2
C3	Convolution	256	27×27	5×5	1
S2	Max pooling	96	27×27	3×3	2
C1	Convolution	96	55×55	11×11	4
In	Input	3 (RGB)	227×227	–	–

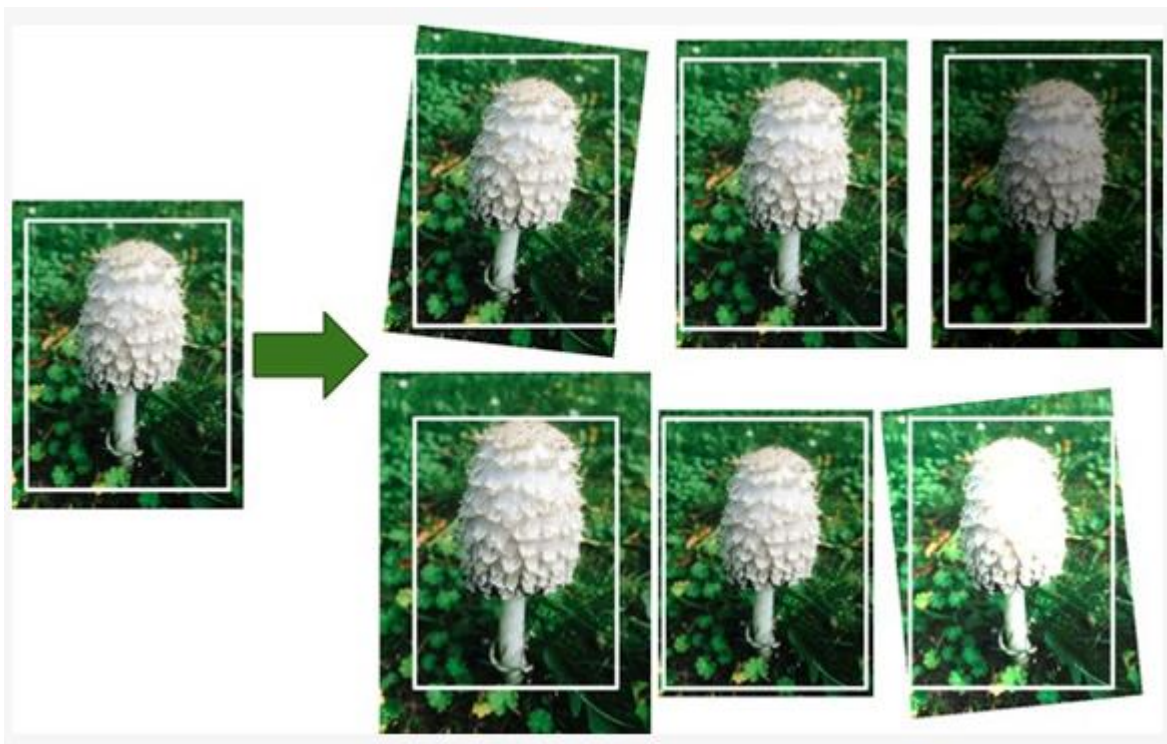
Để giảm overfitting, các tác giả đã sử dụng hai kỹ thuật điều hòa. Đầu tiên, họ áp dụng dropout (được giới thiệu trong Chương 11) với tỷ lệ dropout 50% trong quá trình huấn luyện

cho đầu ra của các lớp F9 và F10. Thứ hai, họ thực hiện tăng cường dữ liệu bằng cách dịch chuyển ngẫu nhiên các hình ảnh huấn luyện bằng các offset khác nhau, lật chúng theo chiều ngang và thay đổi điều kiện ánh sáng.

Tăng cường dữ liệu

Tăng cường dữ liệu làm tăng kích thước của tập huấn luyện một cách nhân tạo bằng cách tạo ra nhiều biến thể thực tế của mỗi phiên bản huấn luyện. Điều này làm giảm overfitting, khiến nó trở thành một kỹ thuật điều hòa. Các phiên bản được tạo ra phải càng thực tế càng tốt: lý tưởng nhất, với một hình ảnh từ tập huấn luyện được tăng cường, một con người không thể phân biệt được nó có được tăng cường hay không. Đơn giản là thêm nhiều trắng sẽ không giúp ích; các sửa đổi phải có thể học được (nhiều trắng thì không).

Ví dụ, bạn có thể dịch chuyển, xoay và thay đổi kích thước nhẹ nhàng mọi bức ảnh trong tập huấn luyện bằng các lượng khác nhau và thêm các bức ảnh thu được vào tập huấn luyện (xem Hình 14-13). Để làm điều này, bạn có thể sử dụng các lớp tăng cường dữ liệu của Keras, được giới thiệu trong Chương 13 (ví dụ: `RandomCrop`, `RandomRotation`, v.v.). Điều này buộc mô hình phải chịu đựng tốt hơn các biến thể về vị trí, hướng và kích thước của các đối tượng trong ảnh. Để tạo ra một mô hình chịu đựng tốt hơn các điều kiện ánh sáng khác nhau, bạn cũng có thể tạo ra nhiều hình ảnh với các độ tương phản khác nhau. Nói chung, bạn cũng có thể lật hình ảnh theo chiều ngang (trừ văn bản và các đối tượng không đối xứng khác). Bằng cách kết hợp các phép biến đổi này, bạn có thể tăng đáng kể kích thước tập huấn luyện của mình.



Hình 14-13. Tạo các phiên bản huấn luyện mới từ các phiên bản hiện có.

Tăng cường dữ liệu cũng hữu ích khi bạn có một tập dữ liệu không cân bằng: bạn có thể sử dụng nó để tạo thêm các mẫu của các lớp ít thường xuyên hơn.

AlexNet cũng sử dụng bước chuẩn hóa cạnh tranh ngay sau bước ReLU của các lớp C1 và C3, được gọi là chuẩn hóa phản hồi cục bộ (LRN): các nơ-ron được kích hoạt mạnh nhất ức chế các nơ-ron khác nằm ở cùng vị trí trong các bản đồ đặc trưng lân cận. Kích hoạt cạnh tranh như vậy đã được quan sát thấy trong các nơ-ron sinh học. Điều này khuyến khích các bản đồ đặc trưng khác nhau chuyên biệt hóa, đẩy chúng ra xa nhau và buộc chúng phải khám phá một phạm vi đặc trưng rộng hơn, cuối cùng cải thiện khả năng khái quát hóa. Phương trình 14-2 cho thấy cách áp dụng LRN.

Phương trình 14-2. Chuẩn hóa phản hồi cục bộ (LRN)

$$b_i = a_i \left(k + \alpha \sum_{j=j_{\text{low}}}^{j_{\text{high}}} a_j^2 \right)^{-\beta} \quad \text{with} \quad \begin{cases} j_{\text{high}} = \min \left(i + \frac{r}{2}, f_n - 1 \right) \\ j_{\text{low}} = \max \left(0, i - \frac{r}{2} \right) \end{cases}$$

Trong phương trình này:

- b_i là đầu ra được chuẩn hóa của nơ-ron nằm trong bản đồ đặc trưng i , tại một hàng u và cột v nào đó (lưu ý rằng trong phương trình này chúng ta chỉ xem xét các nơ-ron nằm ở hàng và cột này, nên u và v không được hiển thị).
- a_i là sự kích hoạt của nơ-ron đó sau bước ReLU, nhưng trước khi chuẩn hóa.
- k , α , β và r là các siêu tham số. k được gọi là bias, và r được gọi là bán kính chiều sâu.
- f_n là số lượng bản đồ đặc trưng.

Ví dụ, nếu $r = 2$ và một nơ-ron có sự kích hoạt mạnh, nó sẽ ức chế sự kích hoạt của các nơ-ron nằm trong các bản đồ đặc trưng ngay phía trên và phía dưới của nó.

Trong AlexNet, các siêu tham số được đặt là: $r = 5$, $\alpha = 0.0001$, $\beta = 0.75$ và $k = 2$. Bạn có thể triển khai bước này bằng cách sử dụng hàm `tf.nn.local_response_normalization()` (mà bạn có thể gói trong một lớp Lambda nếu muốn sử dụng nó trong một mô hình Keras).

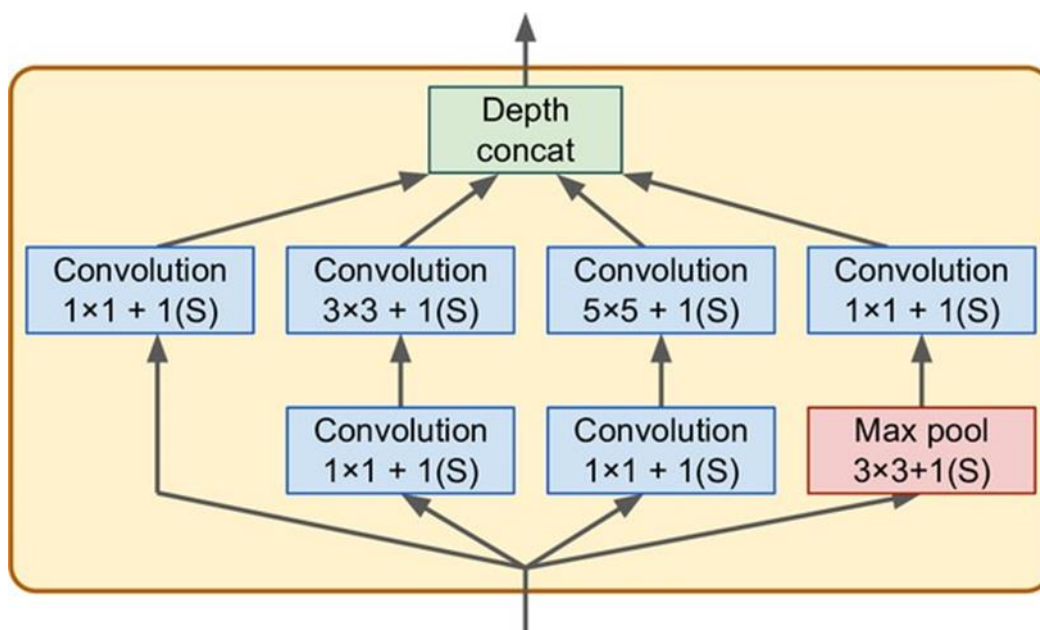
Một biến thể của AlexNet có tên ZF Net được phát triển bởi Matthew Zeiler và Rob Fergus và đã giành chiến thắng thử thách ILSVRC 2013. Về cơ bản, nó là AlexNet với một vài siêu tham số được điều chỉnh (số lượng bản đồ đặc trưng, kích thước kernel, stride, v.v.).

GoogLeNet

Kiến trúc GoogLeNet được phát triển bởi Christian Szegedy et al. từ Google Research, và nó đã giành chiến thắng thử thách ILSVRC 2014 bằng cách đẩy tỷ lệ lỗi top-5 xuống dưới 7%. Hiệu suất tuyệt vời này phần lớn đến từ việc mạng sâu hơn nhiều so với các CNN trước đó (như bạn sẽ thấy trong Hình 14-15). Điều này được thực hiện nhờ các mạng con được gọi là mô-đun inception, cho phép GoogLeNet sử dụng các tham số hiệu quả hơn nhiều so với

các kiến trúc trước đây: GoogLeNet thực sự có số lượng tham số ít hơn 10 lần so với AlexNet (khoảng 6 triệu thay vì 60 triệu).

Hình 14-14 cho thấy kiến trúc của một mô-đun inception. Ký hiệu “ $3 \times 3 + 1(S)$ ” có nghĩa là lớp sử dụng kernel 3×3 , stride 1 và padding “same”. Tín hiệu đầu vào được đưa vào bốn lớp khác nhau song song. Tất cả các lớp tích chập đều sử dụng hàm kích hoạt ReLU. Lưu ý rằng các lớp tích chập trên cùng sử dụng các kích thước kernel khác nhau (1×1 , 3×3 và 5×5), cho phép chúng nắm bắt các mẫu ở các tỷ lệ khác nhau. Cũng lưu ý rằng mỗi lớp đều sử dụng stride là 1 và padding “same” (ngay cả lớp gộp max), vì vậy đầu ra của chúng đều có cùng chiều cao và chiều rộng với đầu vào của chúng. Điều này giúp có thể nối tất cả các đầu ra dọc theo chiều sâu trong lớp nối chiều sâu cuối cùng (tức là xếp chồng các bản đồ đặc trưng từ tất cả bốn lớp tích chập trên cùng). Nó có thể được triển khai bằng cách sử dụng lớp Concatenate của Keras, sử dụng axis=-1 mặc định.



Hình 14-14. Mô-đun Inception.

Bạn có thể tự hỏi tại sao các mô-đun inception lại có các lớp tích chập với kernel 1×1 . Chắc chắn các lớp này không thể nắm bắt bất kỳ đặc trưng nào vì chúng chỉ nhìn vào một pixel tại một thời điểm, phải không? Trên thực tế, các lớp này phục vụ ba mục đích:

- Mặc dù chúng không thể nắm bắt các mẫu không gian, nhưng chúng có thể nắm bắt các mẫu dọc theo chiều sâu (tức là trên các kênh).
- Chúng được cấu hình để xuất ra ít bản đồ đặc trưng hơn đầu vào của chúng, vì vậy chúng đóng vai trò là các lớp thắt cổ chai (bottleneck layers), có nghĩa là chúng giảm chiều. Điều này cắt giảm chi phí tính toán và số lượng tham số, tăng tốc huấn luyện và cải thiện khả năng khái quát hóa.
- Mỗi cặp lớp tích chập ($[1 \times 1, 3 \times 3]$ và $[1 \times 1, 5 \times 5]$) hoạt động như một lớp tích chập mạnh mẽ duy nhất, có khả năng nắm bắt các mẫu phức tạp hơn. Một lớp tích chập

tương đương với việc quét một lớp dày đặc trên hình ảnh (tại mỗi vị trí, nó chỉ nhìn vào một trường tiếp nhận nhỏ), và các cặp lớp tích chập này tương đương với việc quét các mạng nơ-ron hai lớp trên hình ảnh.

Tóm lại, bạn có thể nghĩ toàn bộ mô-đun inception như một lớp tích chập được tăng cường, có khả năng xuất ra các bản đồ đặc trưng nắm bắt các mẫu phức tạp ở các tỷ lệ khác nhau.

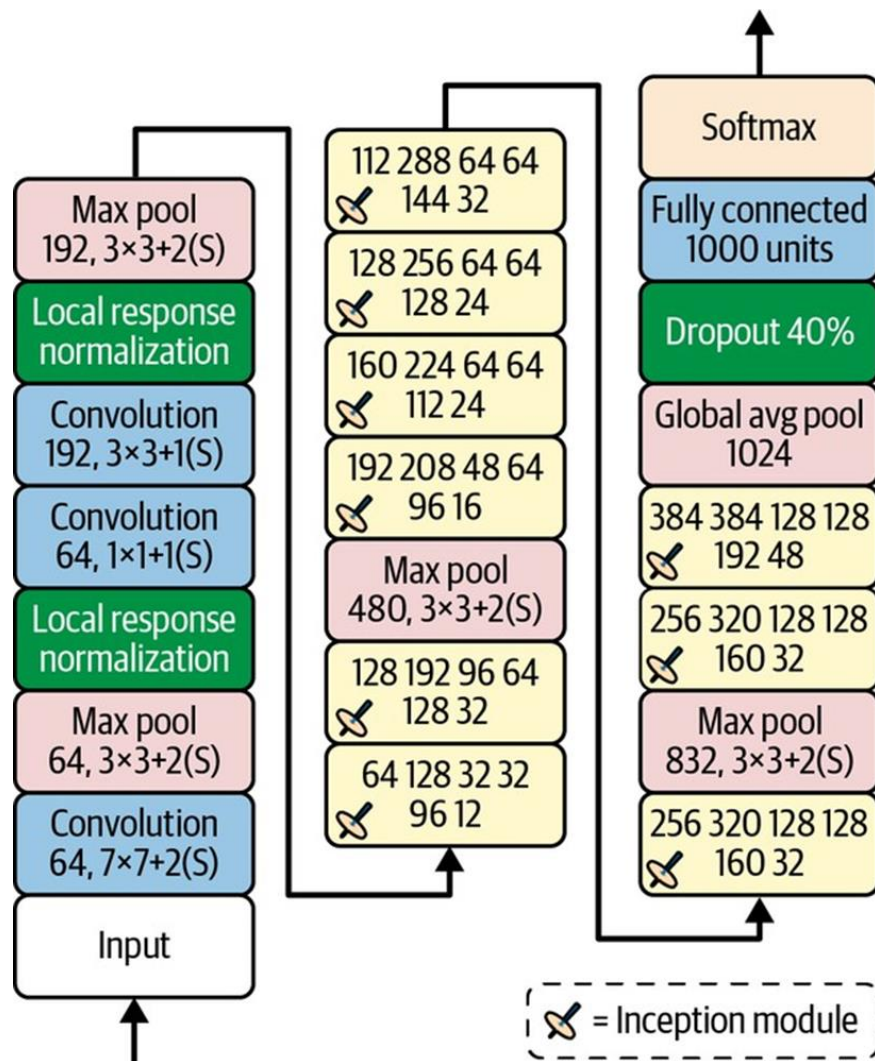
Bây giờ hãy xem kiến trúc của CNN GoogLeNet (xem Hình 14-15).

Số lượng bản đồ đặc trưng được xuất bởi mỗi lớp tích chập và mỗi lớp gộp được hiển thị trước kích thước kernel. Kiến trúc quá sâu đến mức phải được biểu diễn trong ba cột, nhưng GoogLeNet thực sự là một chồng cao, bao gồm chín mô-đun inception (các hộp có đỉnh xoay). Sáu số trong các mô-đun inception đại diện cho số lượng bản đồ đặc trưng được xuất bởi mỗi lớp tích chập trong mô-đun (theo cùng thứ tự như trong Hình 14-14).

Lưu ý rằng tất cả các lớp tích chập đều sử dụng hàm kích hoạt ReLU.

Hãy cùng xem qua mạng này:

- Hai lớp đầu tiên chia chiều cao và chiều rộng của hình ảnh cho 4 (vì vậy diện tích của nó được chia cho 16), để giảm tải tính toán. Lớp đầu tiên sử dụng kích thước kernel lớn, 7×7 , để phần lớn thông tin được bảo toàn.
- Sau đó, lớp chuẩn hóa phản hồi cục bộ đảm bảo rằng các lớp trước học được nhiều loại đặc trưng khác nhau (như đã thảo luận trước đó).
- Hai lớp tích chập tiếp theo, trong đó lớp đầu tiên hoạt động như một lớp thất cổ chai. Như đã đề cập, bạn có thể coi cặp này như một lớp tích chập thông minh hơn duy nhất.
- Một lần nữa, một lớp chuẩn hóa phản hồi cục bộ đảm bảo rằng các lớp trước nắm bắt được nhiều loại mẫu khác nhau.
- Tiếp theo, một lớp gộp max giảm chiều cao và chiều rộng của hình ảnh đi 2, một lần nữa để tăng tốc tính toán.
- Sau đó là xương sống của CNN: một chồng cao gồm chín mô-đun inception, xen kẽ với một vài lớp gộp max để giảm chiều và tăng tốc mạng.
- Tiếp theo, lớp gộp trung bình toàn cục xuất ra giá trị trung bình của mỗi bản đồ đặc trưng: điều này loại bỏ bất kỳ thông tin không gian nào còn lại, điều này không sao vì không còn nhiều thông tin không gian tại thời điểm đó. Thật vậy, hình ảnh đầu vào GoogLeNet thường được dự kiến là 224×224 pixel, vì vậy sau 5 lớp gộp max, mỗi lớp chia chiều cao và chiều rộng cho 2, các bản đồ đặc trưng giảm xuống còn 7×7 . Hơn nữa, đây là một tác vụ phân loại, không phải định vị, vì vậy không quan trọng đối tượng ở đâu. Nhờ việc giảm chiều do lớp này mang lại, không cần phải có nhiều lớp kết nối đầy đủ ở trên cùng của CNN (như trong AlexNet), và điều này làm giảm đáng kể số lượng tham số trong mạng và hạn chế rủi ro quá khớp.
- Các lớp cuối cùng tự giải thích: dropout để điều hòa, sau đó là một lớp kết nối đầy đủ với 1.000 đơn vị (vì có 1.000 lớp) và hàm kích hoạt softmax để xuất ra xác suất lớp ước tính.



Hình 14-15. Kiến trúc GoogLeNet.

Kiến trúc GoogLeNet gốc bao gồm hai bộ phân loại phụ được cắm trên đỉnh của mô-đun inception thứ ba và thứ sáu. Cả hai đều bao gồm một lớp gộp trung bình, một lớp tích chập, hai lớp kết nối đầy đủ và một lớp kích hoạt softmax. Trong quá trình huấn luyện, mất mát của chúng (giảm 70%) được thêm vào mất mát tổng thể. Mục tiêu là chống lại vấn đề vanishing gradients và điều hòa mạng, nhưng sau đó đã chỉ ra rằng hiệu quả của chúng tương đối nhỏ.

Một số biến thể của kiến trúc GoogLeNet sau đó đã được các nhà nghiên cứu của Google đề xuất, bao gồm Inception-v3 và Inception-v4, sử dụng các mô-đun inception hơi khác nhau để đạt được hiệu suất tốt hơn nữa.

VGGNet

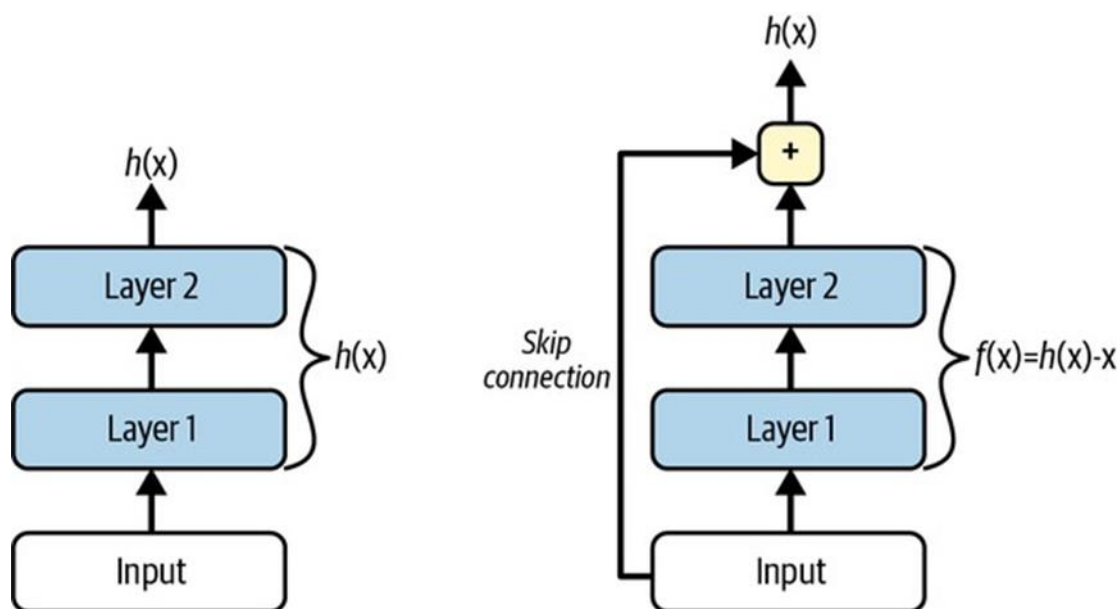
Á quân trong thử thách ILSVRC 2014 là VGGNet, được Karen Simonyan và Andrew Zisserman, từ phòng nghiên cứu Visual Geometry Group (VGG) tại Đại học Oxford, phát triển một kiến trúc rất đơn giản và cổ điển; nó có 2 hoặc 3 lớp tích chập và một lớp gộp, sau

đó lại 2 hoặc 3 lớp tích chập và một lớp gộp, v.v. (đạt tổng cộng 16 hoặc 19 lớp tích chập, tùy thuộc vào biến thể VGG), cộng với một mạng dày đặc cuối cùng với 2 lớp ẩn và lớp đầu ra. Nó sử dụng các bộ lọc 3×3 nhỏ, nhưng có rất nhiều bộ lọc.

ResNet

Kaiming He et al. đã giành chiến thắng thử thách ILSVRC 2015 bằng cách sử dụng Mạng lưới dư thừa (ResNet) với tỷ lệ lỗi top-5 đáng kinh ngạc dưới 3,6%. Biến thể chiến thắng sử dụng một CNN cực kỳ sâu bao gồm 152 lớp (các biến thể khác có 34, 50 và 101 lớp). Nó khẳng định xu hướng chung: các mô hình thị giác máy tính ngày càng sâu hơn, với số lượng tham số ngày càng ít hơn. Chìa khóa để có thể huấn luyện một mạng sâu như vậy là sử dụng các kết nối bỏ qua (skip connections, còn gọi là shortcut connections): tín hiệu đưa vào một lớp cũng được thêm vào đầu ra của một lớp nằm cao hơn trong chồng. Hãy xem tại sao điều này lại hữu ích.

Khi huấn luyện một mạng nơ-ron, mục tiêu là làm cho nó mô hình một hàm mục tiêu $h(x)$. Nếu bạn thêm đầu vào x vào đầu ra của mạng (tức là bạn thêm một kết nối bỏ qua), thì mạng sẽ buộc phải mô hình $f(x) = h(x) - x$ thay vì $h(x)$. Đây được gọi là học dư thừa (residual learning) (xem Hình 14-16).



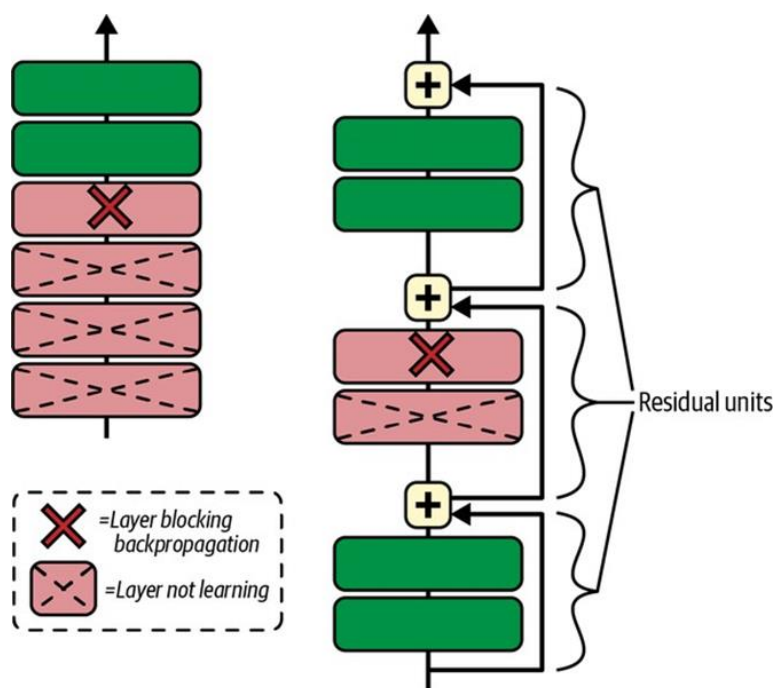
Hình 14-16. Học dư thừa.

Khi bạn khởi tạo một mạng nơ-ron thông thường, các trọng số của nó gần bằng 0, vì vậy mạng chỉ xuất ra các giá trị gần bằng 0. Nếu bạn thêm một kết nối bỏ qua, mạng kết quả chỉ xuất ra một bản sao của đầu vào của nó; nói cách khác, ban đầu nó mô hình hàm đồng nhất. Nếu hàm mục tiêu khá gần với hàm đồng nhất (thường là trường hợp này), điều này sẽ tăng tốc đáng kể quá trình huấn luyện.

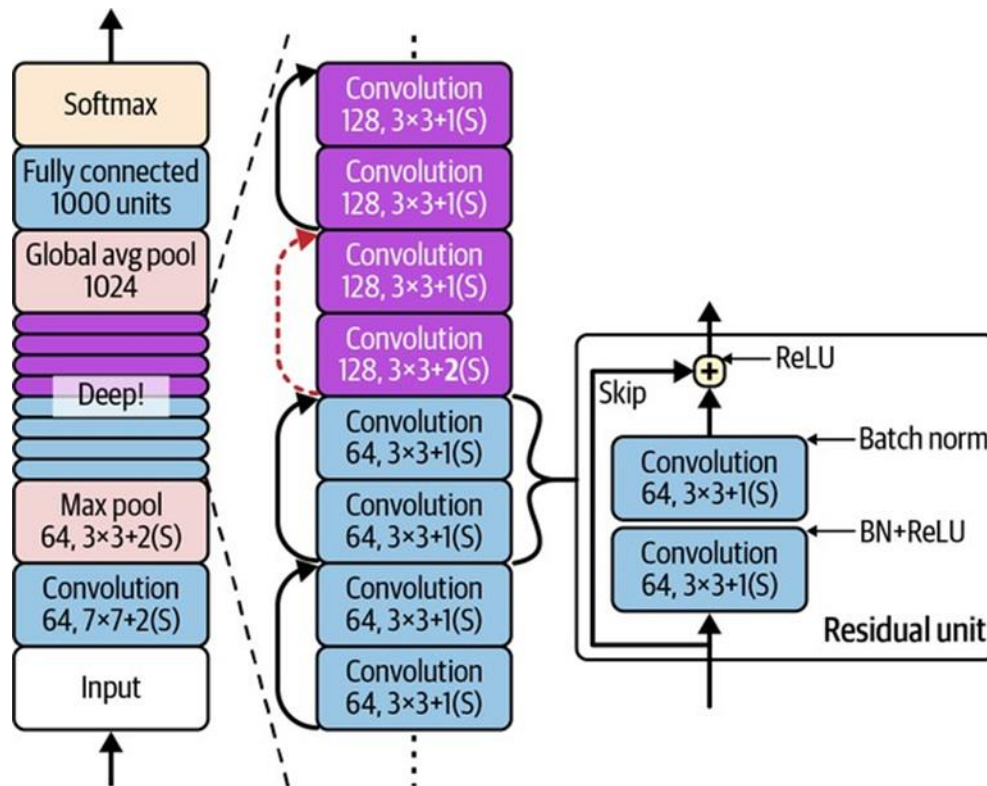
Hơn nữa, nếu bạn thêm nhiều kết nối bỏ qua, mạng có thể bắt đầu tạo ra tiến bộ ngay cả khi một số lớp chưa bắt đầu học (xem Hình 14-17). Nhờ các kết nối bỏ qua, tín hiệu có thể

dễ dàng đi qua toàn bộ mạng. Mạng dư thừa sâu có thể được coi là một chồng các đơn vị dư thừa (RUs), trong đó mỗi đơn vị dư thừa là một mạng nơ-ron nhỏ có kết nối bỏ qua.

Bây giờ hãy xem kiến trúc của ResNet (xem Hình 14-18). Nó đơn giản đáng ngạc nhiên. Nó bắt đầu và kết thúc chính xác như GoogLeNet (ngoại trừ không có lớp dropout), và ở giữa chỉ là một chồng rất sâu các đơn vị dư thừa. Mỗi đơn vị dư thừa bao gồm hai lớp tích chập (và không có lớp gộp!), với chuẩn hóa theo lô (BN) và kích hoạt ReLU, sử dụng kernel 3×3 và bảo toàn các chiều không gian (stride 1, padding “same”).

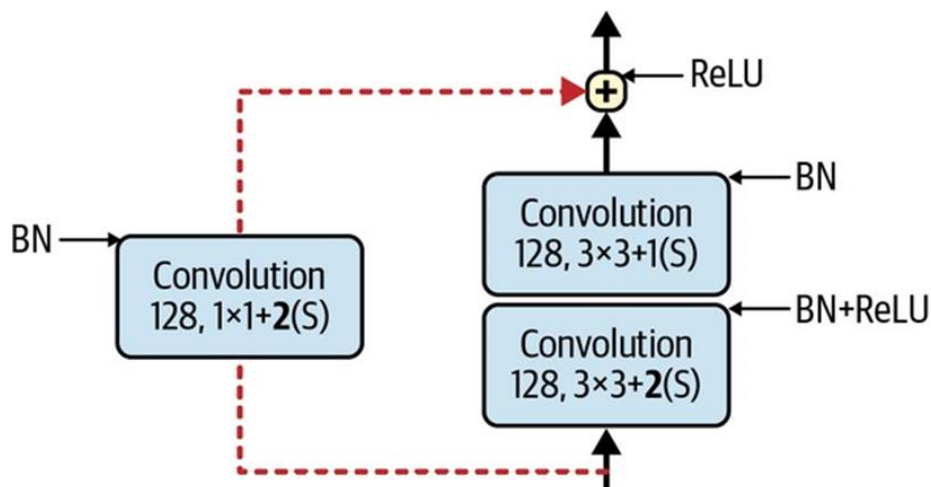


Hình 14-17. Mạng nơ-ron sâu thông thường (trái) và mạng nơ-ron dư thừa sâu (phải).



Hình 14-18. Kiến trúc ResNet.

Lưu ý rằng số lượng bản đồ đặc trưng được tăng gấp đôi sau mỗi vài đơn vị dư thừa, đồng thời chiều cao và chiều rộng của chúng được giảm một nửa (sử dụng một lớp tích chập với stride 2). Khi điều này xảy ra, đầu vào không thể được thêm trực tiếp vào đầu ra của đơn vị dư thừa vì chúng không có cùng hình dạng (ví dụ, vấn đề này ảnh hưởng đến kết nối bỏ qua được biểu thị bằng mũi tên nét đứt trong Hình 14-18). Để giải quyết vấn đề này, đầu vào được truyền qua một lớp tích chập 1×1 với stride 2 và số lượng bản đồ đặc trưng đầu ra phù hợp (xem Hình 14-19).



Hình 14-19. Kết nối bỏ qua khi thay đổi kích thước và chiều sâu bản đồ đặc trưng.

Các biến thể khác nhau của kiến trúc tồn tại, với số lượng lớp khác nhau. ResNet-34 là một ResNet với 34 lớp (chỉ tính các lớp tích chập và lớp kết nối đầy đủ) chứa 3 RU xuất ra 64 bản đồ đặc trưng, 4 RU với 128 bản đồ, 6 RU với 256 bản đồ và 3 RU với 512 bản đồ. Chúng ta sẽ triển khai kiến trúc này sau trong chương này.

Các ResNet sâu hơn, chẳng hạn như ResNet-152, sử dụng các đơn vị dư thừa hơi khác. Thay vì hai lớp tích chập 3×3 với, chẳng hạn, 256 bản đồ đặc trưng, chúng sử dụng ba lớp tích chập: đầu tiên là một lớp tích chập 1×1 với chỉ 64 bản đồ đặc trưng (ít hơn 4 lần), hoạt động như một lớp thắt cổ chai (như đã thảo luận), sau đó là một lớp 3×3 với 64 bản đồ đặc trưng, và cuối cùng là một lớp tích chập 1×1 khác với 256 bản đồ đặc trưng (4 lần 64) để khôi phục chiều sâu ban đầu. ResNet-152 chứa 3 RU như vậy xuất ra 256 bản đồ, sau đó 8 RU với 512 bản đồ, 36 RU khổng lồ với 1.024 bản đồ, và cuối cùng là 3 RU với 2.048 bản đồ.

Xception

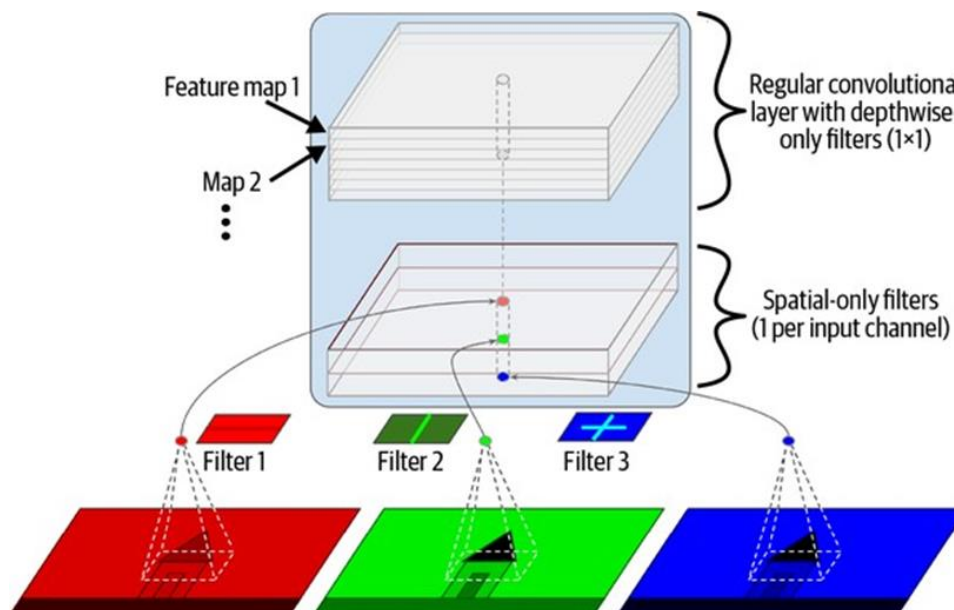
Một biến thể khác của kiến trúc GoogLeNet đáng chú ý: Xception (viết tắt của Extreme Inception) được đề xuất vào năm 2016 bởi François Chollet (tác giả của Keras), và nó đã vượt trội đáng kể so với Inception-v3 trong một tác vụ thị giác khổng lồ (350 triệu hình ảnh và 17.000 lớp). Giống như Inception-v4, nó hợp nhất các ý tưởng của GoogLeNet và ResNet, nhưng nó thay thế các mô-đun inception bằng một loại lớp đặc biệt gọi là lớp tích chập tách biệt chiều sâu (depthwise separable convolution layer, hoặc separable convolution layer viết tắt).

Các lớp này đã được sử dụng trước đây trong một số kiến trúc CNN, nhưng chúng không đóng vai trò trung tâm như trong kiến trúc Xception. Trong khi một lớp tích chập thông thường sử dụng các bộ lọc cố gắng đồng thời nắm bắt các mẫu không gian (ví dụ: hình bầu dục) và các mẫu chéo kênh (ví dụ: miệng + mũi + mắt = khuôn mặt), một lớp tích chập tách biệt đưa ra giả định mạnh mẽ rằng các mẫu không gian và các mẫu chéo kênh có thể được mô hình hóa riêng biệt (xem Hình 14-20). Do đó, nó bao gồm hai phần: phần đầu tiên áp dụng một bộ lọc không gian duy nhất cho mỗi bản đồ đặc trưng đầu vào, sau đó phần thứ hai chỉ tìm kiếm các mẫu chéo kênh—nó chỉ là một lớp tích chập thông thường với các bộ lọc 1×1 .

Vì các lớp tích chập tách biệt chỉ có một bộ lọc không gian trên mỗi kênh đầu vào, bạn nên tránh sử dụng chúng sau các lớp có quá ít kênh, chẳng hạn như lớp đầu vào (thừa nhận rằng đó là những gì Hình 14-20 đại diện, nhưng nó chỉ nhằm mục đích minh họa). Vì lý do này, kiến trúc Xception bắt đầu với 2 lớp tích chập thông thường, nhưng sau đó phần còn lại của kiến trúc chỉ sử dụng các tích chập tách biệt (tổng cộng 34), cộng với một vài lớp gộp max và các lớp cuối cùng thông thường (một lớp gộp trung bình toàn cục và một lớp đầu ra dày đặc).

Bạn có thể tự hỏi tại sao Xception lại được coi là một biến thể của GoogLeNet, vì nó không chứa bất kỳ mô-đun inception nào cả. Vâng, như đã thảo luận trước đó, một mô-đun inception chứa các lớp tích chập với bộ lọc 1×1 : những lớp này chỉ tìm kiếm các mẫu chéo kênh. Tuy nhiên, các lớp tích chập nằm trên chúng là các lớp tích chập thông thường tìm kiếm cả các mẫu không gian và các mẫu chéo kênh. Vì vậy, bạn có thể nghĩ một mô-đun

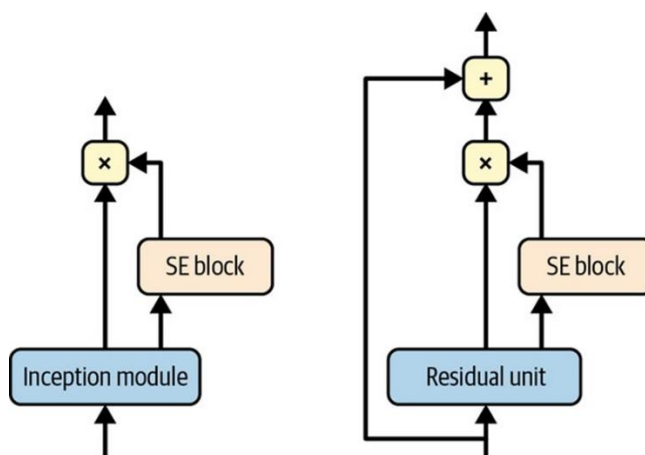
inception là một trung gian giữa một lớp tích chập thông thường (xem xét các mẫu không gian và các mẫu chéo kênh cùng nhau) và một lớp tích chập tách biệt (xem xét chúng riêng biệt). Trong thực tế, có vẻ như các lớp tích chập tách biệt thường hoạt động tốt hơn.



Hình 14-20. Lớp tích chập tách biệt chiều sâu.

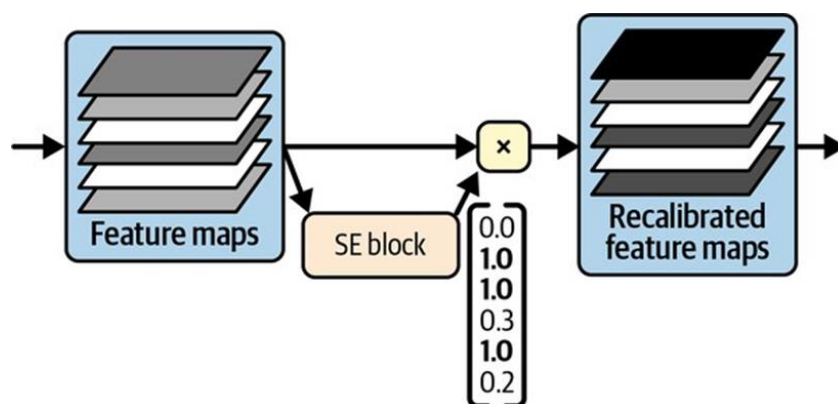
SENet

Kiến trúc chiến thắng trong thử thách ILSVRC 2017 là Mạng nén và kích thích (Squeeze-and-Excitation Network - SENet). Kiến trúc này mở rộng các kiến trúc hiện có như mạng inception và ResNet, và tăng cường hiệu suất của chúng. Điều này cho phép SENet giành chiến thắng trong cuộc thi với tỷ lệ lỗi top-5 đáng kinh ngạc 2,25%! Các phiên bản mở rộng của mạng inception và ResNet được gọi lần lượt là SE-Inception và SE-ResNet. Sự tăng cường đến từ việc SENet thêm một mạng nơ-ron nhỏ, gọi là khối SE, vào mỗi mô-đun inception hoặc đơn vị dư thừa trong kiến trúc gốc, như thể hiện trong Hình 14-21.



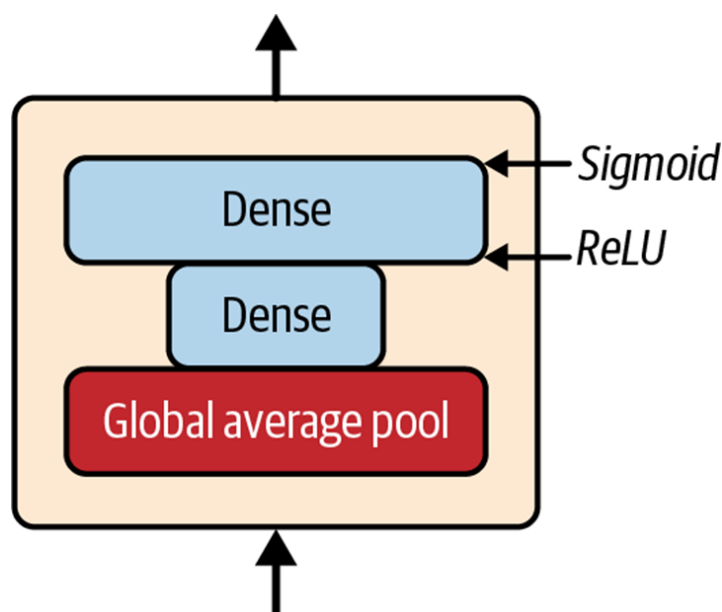
Hình 14-21. Mô-đun SE-Inception (trái) và đơn vị SE-ResNet (phải).

Một khối SE phân tích đầu ra của đơn vị mà nó được gắn vào, chỉ tập trung vào chiều sâu (nó không tìm kiếm bất kỳ mẫu không gian nào), và nó học được các đặc trưng nào thường hoạt động cùng nhau nhất. Sau đó, nó sử dụng thông tin này để hiệu chỉnh lại các bản đồ đặc trưng, như thể hiện trong Hình 14-22. Ví dụ, một khối SE có thể học được rằng miệng, mũi và mắt thường xuất hiện cùng nhau trong các bức ảnh: nếu bạn nhìn thấy miệng và mũi, bạn nên mong đợi cũng thấy mắt. Vì vậy, nếu khối nhìn thấy sự kích hoạt mạnh mẽ trong các bản đồ đặc trưng miệng và mũi, nhưng chỉ kích hoạt nhẹ trong bản đồ đặc trưng mắt, nó sẽ tăng cường bản đồ đặc trưng mắt (chính xác hơn, nó sẽ giảm các bản đồ đặc trưng không liên quan). Nếu mắt bị nhầm lẫn với thứ khác, việc hiệu chỉnh lại bản đồ đặc trưng này sẽ giúp giải quyết sự mơ hồ.



Hình 14-22. Một khối SE thực hiện hiệu chỉnh lại bản đồ đặc trưng.

Một khối SE bao gồm chỉ ba lớp: một lớp gộp trung bình toàn cục, một lớp ẩn dày đặc sử dụng hàm kích hoạt ReLU, và một lớp đầu ra dày đặc sử dụng hàm kích hoạt sigmoid (xem Hình 14-23).



Hình 14-23. Kiến trúc khối SE.

Như trước đây, lớp gộp trung bình toàn cục tính toán sự kích hoạt trung bình cho mỗi bản đồ đặc trưng: ví dụ, nếu đầu vào của nó chứa 256 bản đồ đặc trưng, nó sẽ xuất ra 256 số đại diện cho mức độ phản hồi tổng thể cho mỗi bộ lọc.

Lớp tiếp theo là nơi xảy ra “nén” (squeeze): lớp này có số lượng nơ-ron ít hơn đáng kể so với 256—thường ít hơn 16 lần so với số lượng bản đồ đặc trưng (ví dụ: 16 nơ-ron)—vì vậy 256 số được nén thành một vectơ nhỏ (ví dụ: 16 chiều). Đây là một biểu diễn vectơ chiều thấp (tức là một embedding) của phân bố phản hồi đặc trưng. Bước thắt cổ chai này buộc khối SE phải học một biểu diễn tổng quát của các kết hợp đặc trưng (chúng ta sẽ thấy nguyên tắc này hoạt động trở lại khi thảo luận về autoencoder trong Chương 17). Cuối cùng, lớp đầu ra lấy embedding và xuất ra một vectơ hiệu chỉnh lại chứa một số trên mỗi bản đồ đặc trưng (ví dụ: 256), mỗi số nằm trong khoảng từ 0 đến 1. Các bản đồ đặc trưng sau đó được nhân với vectơ hiệu chỉnh lại này, vì vậy các đặc trưng không liên quan (với điểm hiệu chỉnh lại thấp) sẽ được giảm tỷ lệ trong khi các đặc trưng liên quan (với điểm hiệu chỉnh lại gần 1) được giữ nguyên.

Các kiến trúc đáng chú ý khác

Có nhiều kiến trúc CNN khác để khám phá. Dưới đây là tổng quan ngắn gọn về một số kiến trúc đáng chú ý nhất:

ResNeXt

ResNeXt cải thiện các đơn vị dư thừa trong ResNet. Trong khi các đơn vị dư thừa trong các mô hình ResNet tốt nhất chỉ chứa 3 lớp tích chập mỗi đơn vị, các đơn vị dư thừa ResNeXt bao gồm nhiều chồng song song (ví dụ: 32 chồng), với 3 lớp tích chập mỗi chồng. Tuy nhiên, hai lớp đầu tiên trong mỗi chồng chỉ sử dụng một vài bộ lọc (ví dụ: chỉ bốn), vì vậy tổng số tham số vẫn giữ nguyên như trong ResNet. Sau đó, đầu ra của tất cả các chồng được cộng lại với nhau, và kết quả được truyền đến đơn vị dư thừa tiếp theo (cùng với kết nối bỏ qua).

DenseNet

Một DenseNet bao gồm một số khối dày đặc, mỗi khối được tạo thành từ một vài lớp tích chập được kết nối dày đặc. Kiến trúc này đạt được độ chính xác tuyệt vời trong khi sử dụng tương đối ít tham số. “Kết nối dày đặc” có nghĩa là gì? Đầu ra của mỗi lớp được đưa vào làm đầu vào cho mọi lớp sau nó trong cùng một khối. Ví dụ, lớp 4 trong một khối lấy làm đầu vào sự nối chiều sâu của đầu ra của các lớp 1, 2 và 3 trong khối đó. Các khối dày đặc được phân tách bởi một vài lớp chuyển tiếp.

MobileNet

MobileNet là các mô hình được sắp xếp hợp lý được thiết kế để nhẹ và nhanh, khiến chúng phổ biến trong các ứng dụng di động và web. Chúng dựa trên các lớp tích chập tách biệt chiều sâu, giống như Xception. Các tác giả đã đề xuất một số biến thể, đánh đổi một chút độ chính xác để có các mô hình nhanh hơn và nhỏ hơn.

CSPNet

Mạng bán phần xuyên giai đoạn (CrossStage Partial Network - CSPNet) tương tự như DenseNet, nhưng một phần đầu vào của mỗi khối dày đặc được nối trực tiếp vào đầu ra của khối đó, mà không đi qua khối.

EfficientNet

EfficientNet có thể nói là mô hình quan trọng nhất trong danh sách này. Các tác giả đã đề xuất một phương pháp để mở rộng bất kỳ CNN nào một cách hiệu quả, bằng cách đồng thời tăng chiều sâu (số lượng lớp), chiều rộng (số lượng bộ lọc trên mỗi lớp) và độ phân giải (kích thước hình ảnh đầu vào) một cách có nguyên tắc. Điều này được gọi là mở rộng tổng hợp (compound scaling). Họ đã sử dụng tìm kiếm kiến trúc nơ-ron để tìm một kiến trúc tốt cho phiên bản ImageNet được thu nhỏ (với ít hình ảnh hơn và nhỏ hơn), và sau đó sử dụng mở rộng tổng hợp để tạo ra các phiên bản lớn hơn và lớn hơn của kiến trúc này. Khi các mô hình EfficientNet ra đời, chúng đã vượt trội đáng kể so với tất cả các mô hình hiện có, trên tất cả các ngân sách tính toán, và chúng vẫn nằm trong số các mô hình tốt nhất hiện nay.

Việc hiểu phương pháp mở rộng tổng hợp của EfficientNet rất hữu ích để có được hiểu biết sâu sắc hơn về CNN, đặc biệt nếu bạn cần mở rộng một kiến trúc CNN. Nó dựa trên một phép đo logarit của ngân sách tính toán, được ký hiệu là ϕ : nếu ngân sách tính toán của bạn tăng gấp đôi, thì ϕ tăng thêm 1. Nói cách khác, số lượng phép toán dấu phẩy động có sẵn để huấn luyện tỷ lệ thuận với 2^ϕ . Chiều sâu, chiều rộng và độ phân giải của kiến trúc CNN của bạn phải tỷ lệ theo α^ϕ , β^ϕ và γ^ϕ tương ứng. Các hệ số α , β và γ phải lớn hơn 1, và $\alpha + \beta^2 + \gamma^2$ phải gần bằng 2. Các giá trị tối ưu cho các hệ số này phụ thuộc vào kiến trúc của CNN. Để tìm các giá trị tối ưu cho kiến trúc EfficientNet, các tác giả bắt đầu với một mô hình cơ sở nhỏ (EfficientNetB0), cố định $\phi = 1$, và đơn giản là chạy tìm kiếm lưới: họ tìm thấy $\alpha = 1.2$, $\beta = 1.1$ và $\gamma = 1.1$. Sau đó, họ sử dụng các hệ số này để tạo ra một số kiến trúc lớn hơn, được đặt tên là EfficientNetB1 đến EfficientNetB7, cho các giá trị ϕ tăng dần.

Chọn kiến trúc CNN phù hợp

Với rất nhiều kiến trúc CNN, làm thế nào để bạn chọn kiến trúc tốt nhất cho dự án của mình? Vâng, điều đó phụ thuộc vào điều gì quan trọng nhất đối với bạn: Độ chính xác? Kích thước mô hình (ví dụ: để triển khai trên thiết bị di động)? Tốc độ suy luận trên CPU? Trên GPU?

Bảng 14-3 liệt kê các mô hình đã được huấn luyện sẵn tốt nhất hiện có trong Keras (bạn sẽ thấy cách sử dụng chúng sau trong chương này), được sắp xếp theo kích thước mô hình. Bạn có thể tìm thấy danh sách đầy đủ tại <https://keras.io/api/applications>. Đối với mỗi mô hình, bảng hiển thị tên lớp Keras để sử dụng (trong gói `tf.keras.applications`), kích thước của mô hình tính bằng MB, độ chính xác xác thực top-1 và top-5 trên tập dữ liệu ImageNet, số lượng tham số (triệu), và thời gian suy luận trên CPU và GPU tính bằng ms, sử dụng các lô 32 hình ảnh trên phần cứng đủ mạnh. Đối với mỗi cột, giá trị tốt nhất được làm nổi bật. Như bạn có thể thấy, các mô hình lớn hơn thường chính xác hơn, nhưng không phải lúc nào cũng vậy; ví dụ, EfficientNetB2 vượt trội hơn InceptionV3 cả về kích thước và độ chính xác. Tôi chỉ giữ InceptionV3 trong danh sách vì nó nhanh hơn EfficientNetB2 gần gấp đôi trên

CPU. Tương tự, InceptionResNetV2 nhanh trên CPU, và ResNet50V2 và ResNet101V2 cực kỳ nhanh trên GPU.

Bảng 14-3. Các mô hình đã được huấn luyện sẵn có trong Keras

Tên lớp Keras	Kích thước (MB)	Top-1 Accuracy	Top-5 Accuracy	Parameters (M)	CPU Inference (ms)	GPU Inference (ms)
...	...	...	...	...	...	...
EfficientNetB7	256	84.3%	97.0%	66.7M	1578.9	...
...	...	...	...	...	...	...

Tôi hy vọng bạn thích chuyển đi sâu vào các kiến trúc CNN chính này! Bây giờ hãy xem cách triển khai một trong số chúng bằng Keras.

Triển khai một CNN ResNet-34 bằng Keras

Hầu hết các kiến trúc CNN được mô tả cho đến nay đều có thể được triển khai khá tự nhiên bằng Keras (mặc dù nói chung bạn sẽ tải một mạng đã được huấn luyện sẵn thay vì triển khai thủ công, như bạn sẽ thấy). Để minh họa quá trình này, hãy triển khai một ResNet-34 từ đầu bằng Keras. Đầu tiên, chúng ta sẽ tạo một lớp ResidualUnit:

```
DefaultConv2D = partial(tf.keras.layers.Conv2D, kernel_size=3, strides=1,
                        padding="same", kernel_initializer="he_normal",
                        use_bias=False)
```

```
class ResidualUnit(tf.keras.layers.Layer):
    def __init__(self, filters, strides=1, activation="relu", **kwargs):
        super().__init__(**kwargs)
        self.activation = tf.keras.activations.get(activation)
        self.main_layers = [
            DefaultConv2D(filters, strides=strides),
            tf.keras.layers.BatchNormalization(),
            self.activation,
            DefaultConv2D(filters),
            tf.keras.layers.BatchNormalization()
        ]
        self.skip_layers = []
        if strides > 1:
            self.skip_layers = [
                DefaultConv2D(filters, kernel_size=1, strides=strides),
                tf.keras.layers.BatchNormalization()
            ]

    def call(self, inputs):
        Z = inputs
        for layer in self.main_layers:
```

```

        Z = layer(Z)
    skip_Z = inputs
    for layer in self.skip_layers:
        skip_Z = layer(skip_Z)
    return self.activation(Z + skip_Z)

```

Như bạn có thể thấy, mã này khá khớp với Hình 14-19. Trong hàm khởi tạo, chúng ta tạo tất cả các lớp mà chúng ta sẽ cần: các lớp chính là các lớp ở phía bên phải của sơ đồ, và các lớp bỏ qua là các lớp ở phía bên trái (chỉ cần thiết nếu stride lớn hơn 1). Sau đó, trong phương thức `call()`, chúng ta cho đầu vào đi qua các lớp chính và các lớp bỏ qua (nếu có), và chúng ta cộng cả hai đầu ra và áp dụng hàm kích hoạt.

Bây giờ chúng ta có thể xây dựng một ResNet-34 bằng cách sử dụng mô hình `Sequential`, vì nó thực sự chỉ là một chuỗi dài các lớp—chúng ta có thể coi mỗi đơn vị dư thừa là một lớp duy nhất bây giờ khi chúng ta có lớp `ResidualUnit`. Mã này khớp chặt chẽ với Hình 14-18:

```

model = tf.keras.Sequential([
    DefaultConv2D(64, kernel_size=7, strides=2, input_shape=[224, 224, 3]),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Activation("relu"),
    tf.keras.layers.MaxPool2D(pool_size=3, strides=2, padding="same"),
])
prev_filters = 64

for filters in [64] * 3 + [128] * 4 + [256] * 6 + [512] * 3:
    strides = 1 if filters == prev_filters else 2
    model.add(ResidualUnit(filters, strides=strides))
    prev_filters = filters

model.add(tf.keras.layers.GlobalAvgPool2D())
model.add(tf.keras.layers.Flatten())
model.add(tf.keras.layers.Dense(10, activation="softmax"))

```

Phần khó nhất trong mã này là vòng lặp thêm các lớp `ResidualUnit` vào mô hình: như đã giải thích trước đó, 3 RU đầu tiên có 64 bộ lọc, sau đó 4 RU tiếp theo có 128 bộ lọc, v.v. Ở mỗi lần lặp, chúng ta phải đặt stride thành 1 khi số lượng bộ lọc giống như trong RU trước, hoặc chúng ta đặt nó thành 2; sau đó chúng ta thêm `ResidualUnit`, và cuối cùng chúng ta cập nhật `prev_filters`.

Thật tuyệt vời khi chỉ trong khoảng 40 dòng mã, chúng ta có thể xây dựng mô hình đã giành chiến thắng thử thách ILSVRC 2015! Điều này chứng tỏ cả sự tinh tế của mô hình ResNet và tính biểu cảm của API Keras. Việc triển khai các kiến trúc CNN khác dài hơn một chút, nhưng không khó hơn nhiều. Tuy nhiên, Keras đi kèm với một số kiến trúc này được tích hợp sẵn, vậy tại sao không sử dụng chúng thay thế?

Sử dụng các mô hình đã được huấn luyện sẵn từ Keras

Nói chung, bạn sẽ không phải triển khai các mô hình tiêu chuẩn như GoogLeNet hoặc ResNet theo cách thủ công, vì các mạng đã được huấn luyện sẵn có sẵn chỉ với một dòng mã trong gói `tf.keras.applications`.

Ví dụ, bạn có thể tải mô hình ResNet-50, đã được huấn luyện sẵn trên ImageNet, với dòng mã sau:

```
model = tf.keras.applications.ResNet50(weights="imagenet")
```

Vậy thôi! Điều này sẽ tạo một mô hình ResNet-50 và tải xuống các trọng số đã được huấn luyện sẵn trên tập dữ liệu ImageNet. Để sử dụng nó, trước tiên bạn cần đảm bảo rằng các hình ảnh có kích thước phù hợp. Một mô hình ResNet-50 mong đợi các hình ảnh 224×224 pixel (các mô hình khác có thể mong đợi các kích thước khác, chẳng hạn như 299×299), vì vậy hãy sử dụng lớp Resizing của Keras (được giới thiệu trong Chương 13) để thay đổi kích thước hai hình ảnh mẫu (sau khi cắt chúng theo tỷ lệ khung hình mục tiêu):

```
images = load_sample_images()["images"]
images_resized = tf.keras.layers.Resizing(height=224, width=224,
                                           crop_to_aspect_ratio=True)(images)
```

Các mô hình đã được huấn luyện sẵn giả định rằng các hình ảnh được tiền xử lý theo một cách cụ thể. Trong một số trường hợp, chúng có thể mong đợi đầu vào được chia tỷ lệ từ 0 đến 1, hoặc từ -1 đến 1, v.v. Mỗi mô hình cung cấp một hàm `preprocess_input()` mà bạn có thể sử dụng để tiền xử lý hình ảnh của mình. Các hàm này giả định rằng các giá trị pixel gốc nằm trong khoảng từ 0 đến 255, đây là trường hợp ở đây:

```
inputs = tf.keras.applications.resnet50.preprocess_input(images_resized)
```

Bây giờ chúng ta có thể sử dụng mô hình đã được huấn luyện sẵn để đưa ra dự đoán:

```
>>> Y_proba = model.predict(inputs)
>>> Y_proba.shape
(2, 1000)
```

Như thường lệ, đầu ra `Y_proba` là một ma trận với một hàng trên mỗi hình ảnh và một cột trên mỗi lớp (trong trường hợp này, có 1.000 lớp). Nếu bạn muốn hiển thị K dự đoán hàng đầu, bao gồm tên lớp và xác suất ước tính của mỗi lớp được dự đoán, hãy sử dụng hàm `decode_predictions()`. Đối với mỗi hình ảnh, nó trả về một mảng chứa K dự đoán hàng đầu, trong đó mỗi dự đoán được biểu diễn dưới dạng một mảng chứa định danh lớp, tên của nó và điểm tin cậy tương ứng:

```
top_K = tf.keras.applications.resnet50.decode_predictions(Y_proba, top=3)
for image_index in range(len(images)):
    print(f"Image #{image_index}")
    for class_id, name, y_proba in top_K[image_index]:
        print(f" {class_id} - {name:12s} {y_proba:.2%}")
```

Đầu ra trông như thế này:

```

Image #0
n03877845 - palace      54.69%
n03781244 - monastery   24.72%
n02825657 - bell_cote   18.55%
Image #1
n04522168 - vase        32.66%
n11939491 - daisy       17.81%
n03530642 - honeycomb   12.06%

```

Các lớp chính xác là palace và dahlia, vì vậy mô hình đúng cho hình ảnh đầu tiên nhưng sai cho hình ảnh thứ hai. Tuy nhiên, đó là do dahlia không phải là một trong 1.000 lớp ImageNet. Với điều đó, vase là một phỏng đoán hợp lý (có lẽ bông hoa nằm trong một chiếc bình?), và daisy cũng không phải là một lựa chọn tồi, vì dahlia và daisy đều thuộc cùng họ Compositae.

Như bạn có thể thấy, rất dễ dàng để tạo một bộ phân loại hình ảnh khá tốt bằng cách sử dụng một mô hình đã được huấn luyện sẵn. Như bạn đã thấy trong Bảng 14-3, nhiều mô hình thị giác khác có sẵn trong `tf.keras.applications`, từ các mô hình nhẹ và nhanh đến các mô hình lớn và chính xác. Nhưng điều gì sẽ xảy ra nếu bạn muốn sử dụng bộ phân loại hình ảnh cho các lớp hình ảnh không phải là một phần của ImageNet? Trong trường hợp đó, bạn vẫn có thể hưởng lợi từ các mô hình đã được huấn luyện sẵn bằng cách sử dụng chúng để thực hiện học chuyển đổi.

Học chuyển đổi với các mô hình đã được huấn luyện sẵn

Nếu bạn muốn xây dựng một bộ phân loại hình ảnh nhưng không có đủ dữ liệu để huấn luyện nó từ đầu, thì thường là một ý kiến hay khi tái sử dụng các lớp thấp hơn của một mô hình đã được huấn luyện sẵn, như chúng ta đã thảo luận trong Chương 11.

Ví dụ, hãy huấn luyện một mô hình để phân loại hình ảnh hoa, tái sử dụng một mô hình Xception đã được huấn luyện sẵn. Đầu tiên, chúng ta sẽ tải tập dữ liệu hoa bằng cách sử dụng TensorFlow Datasets (được giới thiệu trong Chương 13):

```

import tensorflow_datasets as tfds

dataset, info = tfds.load("tf_flowers", as_supervised=True, with_info=True)
dataset_size = info.splits["train"].num_examples # 3670
class_names = info.features["label"].names # ["dandelion", "daisy", ...]
n_classes = info.features["label"].num_classes # 5

```

Lưu ý rằng bạn có thể nhận thông tin về tập dữ liệu bằng cách đặt `with_info=True`. Ở đây, chúng ta nhận được kích thước tập dữ liệu và tên của các lớp. Thật không may, chỉ có tập dữ liệu “train”, không có tập kiểm tra hoặc tập xác thực, vì vậy chúng ta cần chia tập huấn luyện. Hãy gọi `tfds.load()` một lần nữa, nhưng lần này lấy 10% đầu tiên của tập dữ liệu để kiểm tra, 15% tiếp theo để xác thực và 75% còn lại để huấn luyện:

```

test_set_raw, valid_set_raw, train_set_raw = tfds.load(
    "tf_flowers",
    split=["train[:10%]", "train[10%:25%]", "train[25%:]"],

```

```
as_supervised=True
)
```

Tất cả ba tập dữ liệu đều chứa các hình ảnh riêng lẻ. Chúng ta cần nhóm chúng lại (batch), nhưng trước tiên chúng ta cần đảm bảo tất cả chúng có cùng kích thước, nếu không việc nhóm sẽ thất bại. Chúng ta có thể sử dụng lớp Resizing cho việc này. Chúng ta cũng phải gọi hàm `tf.keras.applications.xception.preprocess_input()` để tiền xử lý hình ảnh một cách thích hợp cho mô hình Xception. Cuối cùng, chúng ta cũng sẽ xáo trộn tập huấn luyện và sử dụng prefetching:

```
batch_size = 32
preprocess = tf.keras.Sequential([
    tf.keras.layers.Resizing(height=224, width=224,
                             crop_to_aspect_ratio=True),
    tf.keras.layers.Lambda(tf.keras.applications.xception.preprocess_input)
])

train_set = train_set_raw.map(lambda X, y: (preprocess(X), y))
train_set = train_set.shuffle(1000, seed=42).batch(batch_size).prefetch(1)
valid_set = valid_set_raw.map(lambda X, y: (preprocess(X),
y)).batch(batch_size)
test_set = test_set_raw.map(lambda X, y: (preprocess(X),
y)).batch(batch_size)
```

Bây giờ mỗi lô chứa 32 hình ảnh, tất cả đều có kích thước 224×224 pixel, với các giá trị pixel trong khoảng từ -1 đến 1. Hoàn hảo!

Vì tập dữ liệu không quá lớn, một chút tăng cường dữ liệu chắc chắn sẽ hữu ích. Hãy tạo một mô hình tăng cường dữ liệu mà chúng ta sẽ nhúng vào mô hình cuối cùng của mình. Trong quá trình huấn luyện, nó sẽ ngẫu nhiên lật hình ảnh theo chiều ngang, xoay chúng một chút và điều chỉnh độ tương phản:

```
data_augmentation = tf.keras.Sequential([
    tf.keras.layers.RandomFlip(mode="horizontal", seed=42),
    tf.keras.layers.RandomRotation(factor=0.05, seed=42),
    tf.keras.layers.RandomContrast(factor=0.2, seed=42)
])
```

Tiếp theo, hãy tải một mô hình Xception, đã được huấn luyện sẵn trên ImageNet. Chúng ta loại trừ phần trên cùng của mạng bằng cách đặt `include_top=False`. Điều này loại trừ lớp gộp trung bình toàn cục và lớp đầu ra dày đặc. Sau đó, chúng ta thêm lớp gộp trung bình toàn cục của riêng mình (cấp cho nó đầu ra của mô hình cơ sở), theo sau bởi một lớp đầu ra dày đặc với một đơn vị trên mỗi lớp, sử dụng hàm kích hoạt softmax.

Cuối cùng, chúng ta gói tất cả điều này trong một Keras Model:

```
import tensorflow as tf

base_model = tf.keras.applications.xception.Xception(weights="imagenet",
```

```

                                include_top=False)
avg = tf.keras.layers.GlobalAveragePooling2D()(base_model.output)
output = tf.keras.layers.Dense(n_classes, activation="softmax")(avg)
model = tf.keras.Model(inputs=base_model.input, outputs=output)

```

Như đã giải thích trong Chương 11, thường là một ý kiến hay khi đóng băng các trọng số của các lớp đã được huấn luyện sẵn, ít nhất là ở đầu quá trình huấn luyện:

```

for layer in base_model.layers:
    layer.trainable = False

```

Cuối cùng, chúng ta có thể biên dịch mô hình và bắt đầu huấn luyện:

```

optimizer = tf.keras.optimizers.SGD(learning_rate=0.1, momentum=0.9)
model.compile(loss="sparse_categorical_crossentropy", optimizer=optimizer,
              metrics=["accuracy"])
history = model.fit(train_set, validation_data=valid_set, epochs=3)

```

Sau khi huấn luyện mô hình trong vài epoch, độ chính xác thực của nó sẽ đạt hơn 80% một chút và sau đó ngừng cải thiện. Điều này có nghĩa là các lớp trên cùng hiện đã được huấn luyện khá tốt, và chúng ta đã sẵn sàng bỏ đóng băng một số lớp trên cùng của mô hình cơ sở, sau đó tiếp tục huấn luyện. Ví dụ, hãy bỏ đóng băng các lớp 56 trở lên (đó là khởi đầu của đơn vị dư thừa 7 trên 14, như bạn có thể thấy nếu liệt kê tên các lớp):

```

for layer in base_model.layers[56:]:
    layer.trainable = True

```

Đừng quên biên dịch mô hình mỗi khi bạn đóng băng hoặc bỏ đóng băng các lớp. Ngoài ra, hãy đảm bảo sử dụng tốc độ học thấp hơn nhiều để tránh làm hỏng các trọng số đã được huấn luyện sẵn:

```

optimizer = tf.keras.optimizers.SGD(learning_rate=0.01, momentum=0.9)
model.compile(loss="sparse_categorical_crossentropy", optimizer=optimizer,
              metrics=["accuracy"])
history = model.fit(train_set, validation_data=valid_set, epochs=10)

```

Mô hình này sẽ đạt khoảng 92% độ chính xác trên tập kiểm tra, chỉ trong vài phút huấn luyện (với GPU). Nếu bạn điều chỉnh các siêu tham số, giảm tốc độ học và huấn luyện lâu hơn một chút, bạn sẽ có thể đạt 95% đến 97%. Với điều đó, bạn có thể bắt đầu huấn luyện các bộ phân loại hình ảnh tuyệt vời trên hình ảnh và các lớp của riêng mình! Nhưng thị giác máy tính còn nhiều điều hơn là chỉ phân loại. Ví dụ, điều gì sẽ xảy ra nếu bạn cũng muốn biết bông hoa nằm ở đâu trong một bức ảnh? Hãy xem xét điều này ngay bây giờ.

Phân loại và định vị

Định vị một đối tượng trong một bức ảnh có thể được biểu thị dưới dạng một tác vụ hồi quy, như đã thảo luận trong Chương 10: để dự đoán một hộp bao quanh đối tượng, một cách tiếp cận phổ biến là dự đoán tọa độ ngang và dọc của tâm đối tượng, cũng như chiều cao và chiều rộng của nó. Điều này có nghĩa là chúng ta có bốn số cần dự đoán. Nó không yêu cầu nhiều thay đổi đối với mô hình; chúng ta chỉ cần thêm một lớp đầu ra dày đặc thứ hai

với bốn đơn vị (thường nằm trên lớp gộp trung bình toàn cục), và nó có thể được huấn luyện bằng hàm mất mát MSE:

```
base_model = tf.keras.applications.xception.Xception(weights="imagenet",
                                                    include_top=False)
avg = tf.keras.layers.GlobalAveragePooling2D()(base_model.output)
class_output = tf.keras.layers.Dense(n_classes, activation="softmax")(avg)
loc_output = tf.keras.layers.Dense(4)(avg) # 4 units for x, y, width, height
model = tf.keras.Model(inputs=base_model.input,
                       outputs=[class_output, loc_output])
model.compile(loss=["sparse_categorical_crossentropy", "mse"],
              loss_weights=[0.8, 0.2], # depends on what you care most about
              optimizer=optimizer, metrics=["accuracy"])
```

Nhưng bây giờ chúng ta có một vấn đề: tập dữ liệu hoa không có hộp bao quanh các bông hoa. Vì vậy, chúng ta cần tự thêm chúng vào. Đây thường là một trong những phần khó khăn và tốn kém nhất của một dự án học máy: lấy nhãn. Nên dành thời gian tìm kiếm các công cụ phù hợp. Để chú thích hình ảnh với hộp bao quanh, bạn có thể muốn sử dụng một công cụ gắn nhãn hình ảnh mã nguồn mở như VGG Image Annotator, LabelImg, OpenLabeler, hoặc ImgLab, hoặc có lẽ một công cụ thương mại như LabelBox hoặc Supervisely. Bạn cũng có thể muốn xem xét các nền tảng crowdsourcing như Amazon Mechanical Turk nếu bạn có số lượng hình ảnh rất lớn cần chú thích. Tuy nhiên, việc thiết lập một nền tảng crowdsourcing, chuẩn bị biểu mẫu để gửi cho người lao động, giám sát họ và đảm bảo chất lượng của các hộp bao quanh mà họ tạo ra là tốt, là khá nhiều công việc, vì vậy hãy đảm bảo rằng nó đáng công sức.

Adriana Kovashka và cộng sự đã viết một bài báo rất thực tế về crowdsourcing trong thị giác máy tính. Tôi khuyên bạn nên xem nó, ngay cả khi bạn không có kế hoạch sử dụng crowdsourcing. Nếu chỉ có vài trăm hoặc thậm chí vài nghìn hình ảnh cần gắn nhãn, và bạn không có kế hoạch làm điều này thường xuyên, có thể tốt hơn là tự làm: với các công cụ phù hợp, nó sẽ chỉ mất vài ngày, và bạn cũng sẽ hiểu rõ hơn về tập dữ liệu và nhiệm vụ của mình.

Bây giờ, giả sử bạn đã có được các hộp bao quanh cho mỗi hình ảnh trong tập dữ liệu hoa (hiện tại chúng ta sẽ giả định có một hộp bao quanh duy nhất trên mỗi hình ảnh). Sau đó, bạn cần tạo một tập dữ liệu có các mục sẽ là các lô hình ảnh đã được tiền xử lý cùng với nhãn lớp và hộp bao quanh của chúng. Mỗi mục phải là một tuple có dạng (images, (class_labels, bounding_boxes)). Sau đó, bạn đã sẵn sàng để huấn luyện mô hình của mình!

MSE thường hoạt động khá tốt như một hàm chi phí để huấn luyện mô hình, nhưng nó không phải là một thước đo tuyệt vời để đánh giá mức độ mô hình có thể dự đoán hộp bao quanh. Thước đo phổ biến nhất cho điều này là Intersection over Union (IoU): diện tích chồng chéo giữa hộp bao quanh được dự đoán và hộp bao quanh mục tiêu, chia cho diện tích hợp của chúng (xem Hình 14-24). Trong Keras, nó được triển khai bởi lớp `tf.keras.metrics.MeanIoU`.

Phân loại và định vị một đối tượng duy nhất là tốt, nhưng điều gì sẽ xảy ra nếu hình ảnh chứa nhiều đối tượng (như thường xảy ra trong tập dữ liệu hoa)?

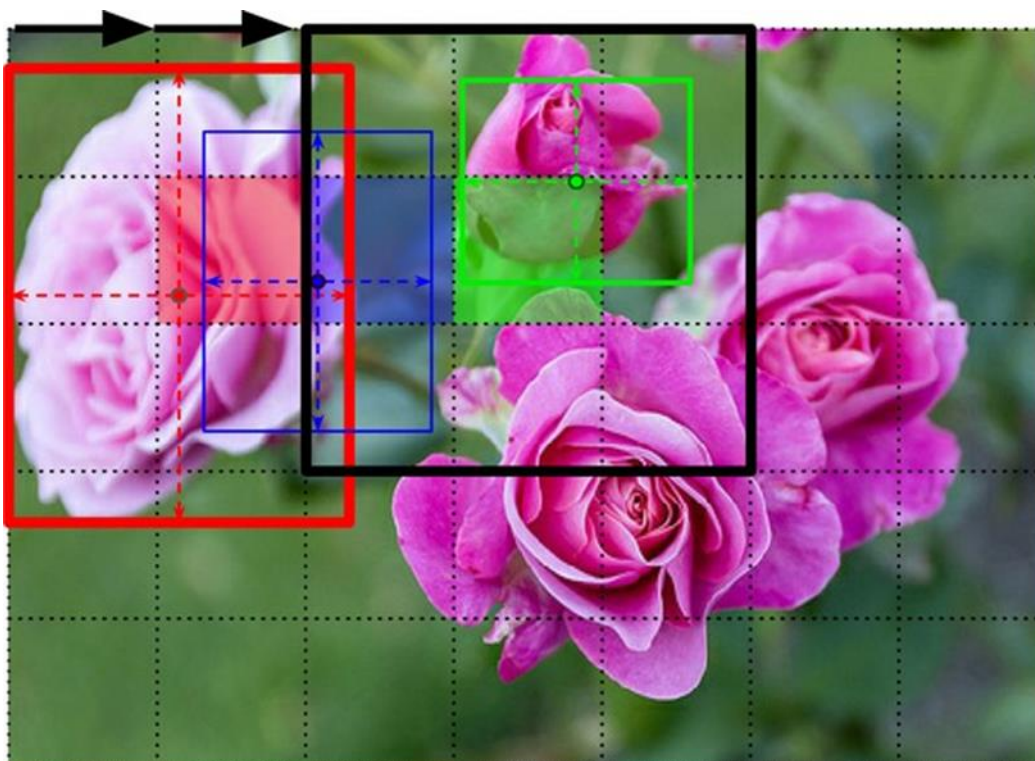


Hình 14-24. Thước đo IoU cho hộp bao quanh.

Phát hiện đối tượng

Nhiệm vụ phân loại và định vị nhiều đối tượng trong một hình ảnh được gọi là phát hiện đối tượng. Cho đến vài năm trước, một cách tiếp cận phổ biến là lấy một CNN đã được huấn luyện để phân loại và định vị một đối tượng duy nhất nằm gần giữa hình ảnh, sau đó trượt CNN này trên hình ảnh và đưa ra dự đoán ở mỗi bước. CNN thường được huấn luyện để dự đoán không chỉ xác suất lớp và một hộp bao quanh, mà còn cả điểm đối tượng (objectness score): đây là xác suất ước tính rằng hình ảnh thực sự chứa một đối tượng nằm gần giữa. Đây là một đầu ra phân loại nhị phân; nó có thể được tạo ra bởi một lớp đầu ra dày đặc với một đơn vị duy nhất, sử dụng hàm kích hoạt sigmoid và được huấn luyện bằng hàm mất mát cross-entropy nhị phân.

Cách tiếp cận CNN trượt này được minh họa trong Hình 14-25. Trong ví dụ này, hình ảnh được chia thành một lưới 5×7 , và chúng ta thấy một CNN—hình chữ nhật đen dày—trượt qua tất cả các vùng 3×3 và đưa ra dự đoán ở mỗi bước.



Hình 14-25. Phát hiện nhiều đối tượng bằng cách trượt CNN trên hình ảnh.

Trong hình này, CNN đã đưa ra dự đoán cho ba trong số các vùng 3×3 này:

- Khi nhìn vào vùng 3×3 phía trên bên trái (tâm ở ô lưới màu đỏ nằm ở hàng thứ hai và cột thứ hai), nó đã phát hiện ra bông hồng ngoài cùng bên trái. Lưu ý rằng hộp bao quanh được dự đoán vượt quá ranh giới của vùng 3×3 này. Điều đó hoàn toàn bình thường: mặc dù CNN không thể nhìn thấy phần dưới của bông hồng, nhưng nó vẫn có thể đưa ra một phỏng đoán hợp lý về vị trí của nó. Nó cũng dự đoán xác suất lớp, cho xác suất cao cho lớp “rose”. Cuối cùng, nó dự đoán điểm đối tượng khá cao, vì tâm của hộp bao quanh nằm trong ô lưới trung tâm (trong hình này, điểm đối tượng được biểu thị bằng độ dày của hộp bao quanh).
- Khi nhìn vào vùng 3×3 tiếp theo, một ô lưới sang phải (tâm ở ô vuông màu xanh lam được tô bóng), nó không phát hiện bất kỳ bông hoa nào có tâm trong vùng đó, vì vậy nó dự đoán điểm đối tượng rất thấp; do đó, hộp bao quanh và xác suất lớp được dự đoán có thể bỏ qua một cách an toàn. Bạn có thể thấy rằng hộp bao quanh được dự đoán không tốt chút nào.
- Cuối cùng, khi nhìn vào vùng 3×3 tiếp theo, một lần nữa một ô lưới sang phải (tâm ở ô màu xanh lá cây được tô bóng), nó đã phát hiện bông hồng ở phía trên, mặc dù không hoàn hảo: bông hồng này không được căn giữa tốt trong vùng này, vì vậy điểm đối tượng được dự đoán không cao lắm.

Bạn có thể hình dung việc trượt CNN trên toàn bộ hình ảnh sẽ cho bạn tổng cộng 15 hộp bao quanh được dự đoán, được sắp xếp trong một lưới 3×5 , với mỗi hộp bao quanh kèm theo xác suất lớp ước tính và điểm đối tượng của nó. Vì các đối tượng có thể có kích thước

khác nhau, bạn có thể muốn trượt lại CNN trên các vùng 4×4 lớn hơn, để có được nhiều hộp bao quanh hơn.

Kỹ thuật này khá đơn giản, nhưng như bạn có thể thấy nó sẽ thường phát hiện cùng một đối tượng nhiều lần, ở các vị trí hơi khác nhau. Cần có một số xử lý hậu kỳ để loại bỏ tất cả các hộp bao quanh không cần thiết. Một cách tiếp cận phổ biến cho việc này được gọi là **non-max suppression**. Đây là cách nó hoạt động:

1. Đầu tiên, loại bỏ tất cả các hộp bao quanh mà điểm đối tượng thấp hơn một ngưỡng nào đó: vì CNN tin rằng không có đối tượng ở vị trí đó, hộp bao quanh là vô dụng.
2. Tìm hộp bao quanh còn lại có điểm đối tượng cao nhất, và loại bỏ tất cả các hộp bao quanh còn lại khác chồng lấn nhiều với nó (ví dụ: với IoU lớn hơn 60%). Ví dụ, trong Hình 14-25, hộp bao quanh có điểm đối tượng max là hộp bao quanh dày trên bông hồng ngoài cùng bên trái. Hộp bao quanh khác chạm vào cùng bông hồng này chồng lấn nhiều với hộp bao quanh max, vì vậy chúng ta sẽ loại bỏ nó (mặc dù trong ví dụ này nó đã được loại bỏ ở bước trước).
3. Lặp lại bước 2 cho đến khi không còn hộp bao quanh nào để loại bỏ.

Cách tiếp cận đơn giản này để phát hiện đối tượng hoạt động khá tốt, nhưng nó yêu cầu chạy CNN nhiều lần (15 lần trong ví dụ này), vì vậy nó khá chậm. May mắn thay, có một cách nhanh hơn nhiều để trượt CNN trên một hình ảnh: sử dụng **mạng hoàn toàn tích chập (FCN)**.

Mạng hoàn toàn tích chập

Ý tưởng về FCN lần đầu tiên được giới thiệu trong một bài báo năm 2015 của Jonathan Long et al., cho phân đoạn ngữ nghĩa (nhiệm vụ phân loại mọi pixel trong một hình ảnh theo lớp của đối tượng mà pixel đó thuộc về). Các tác giả đã chỉ ra rằng bạn có thể thay thế các lớp dày đặc ở trên cùng của một CNN bằng các lớp tích chập. Để hiểu điều này, hãy xem một ví dụ: giả sử một lớp dày đặc với 200 nơ-ron nằm trên một lớp tích chập xuất ra 100 bản đồ đặc trưng, mỗi bản đồ có kích thước 7×7 (đây là kích thước bản đồ đặc trưng, không phải kích thước kernel). Mỗi nơ-ron sẽ tính tổng trọng số của tất cả $100 \times 7 \times 7$ kích hoạt từ lớp tích chập (cộng với một thuật ngữ bias). Bây giờ hãy xem điều gì xảy ra nếu chúng ta thay thế lớp dày đặc bằng một lớp tích chập sử dụng 200 bộ lọc, mỗi bộ lọc có kích thước 7×7 , và với padding “valid”. Lớp này sẽ xuất ra 200 bản đồ đặc trưng, mỗi bản đồ có kích thước 1×1 (vì kernel có kích thước chính xác bằng bản đồ đặc trưng đầu vào và chúng ta đang sử dụng padding “valid”). Nói cách khác, nó sẽ xuất ra 200 số, giống như lớp dày đặc đã làm; và nếu bạn nhìn kỹ vào các phép tính được thực hiện bởi một lớp tích chập, bạn sẽ nhận thấy rằng các số này sẽ chính xác giống như những gì lớp dày đặc đã tạo ra. Điểm khác biệt duy nhất là đầu ra của lớp dày đặc là một tensor có hình dạng [kích thước lô, 200], trong khi lớp tích chập sẽ xuất ra một tensor có hình dạng [kích thước lô, 1, 1, 200].

Tại sao điều này lại quan trọng? Vâng, trong khi một lớp dày đặc mong đợi một kích thước đầu vào cụ thể (vì nó có một trọng số trên mỗi tính năng đầu vào), một lớp tích chập sẽ vui vẻ xử lý hình ảnh ở bất kỳ kích thước nào (tuy nhiên, nó mong đợi đầu vào của nó có một số lượng kênh cụ thể, vì mỗi kernel chứa một tập hợp trọng số khác nhau cho mỗi kênh đầu

vào). Vì FCN chỉ chứa các lớp tích chập (và các lớp gộp, có cùng thuộc tính), nó có thể được huấn luyện và thực thi trên hình ảnh ở bất kỳ kích thước nào!

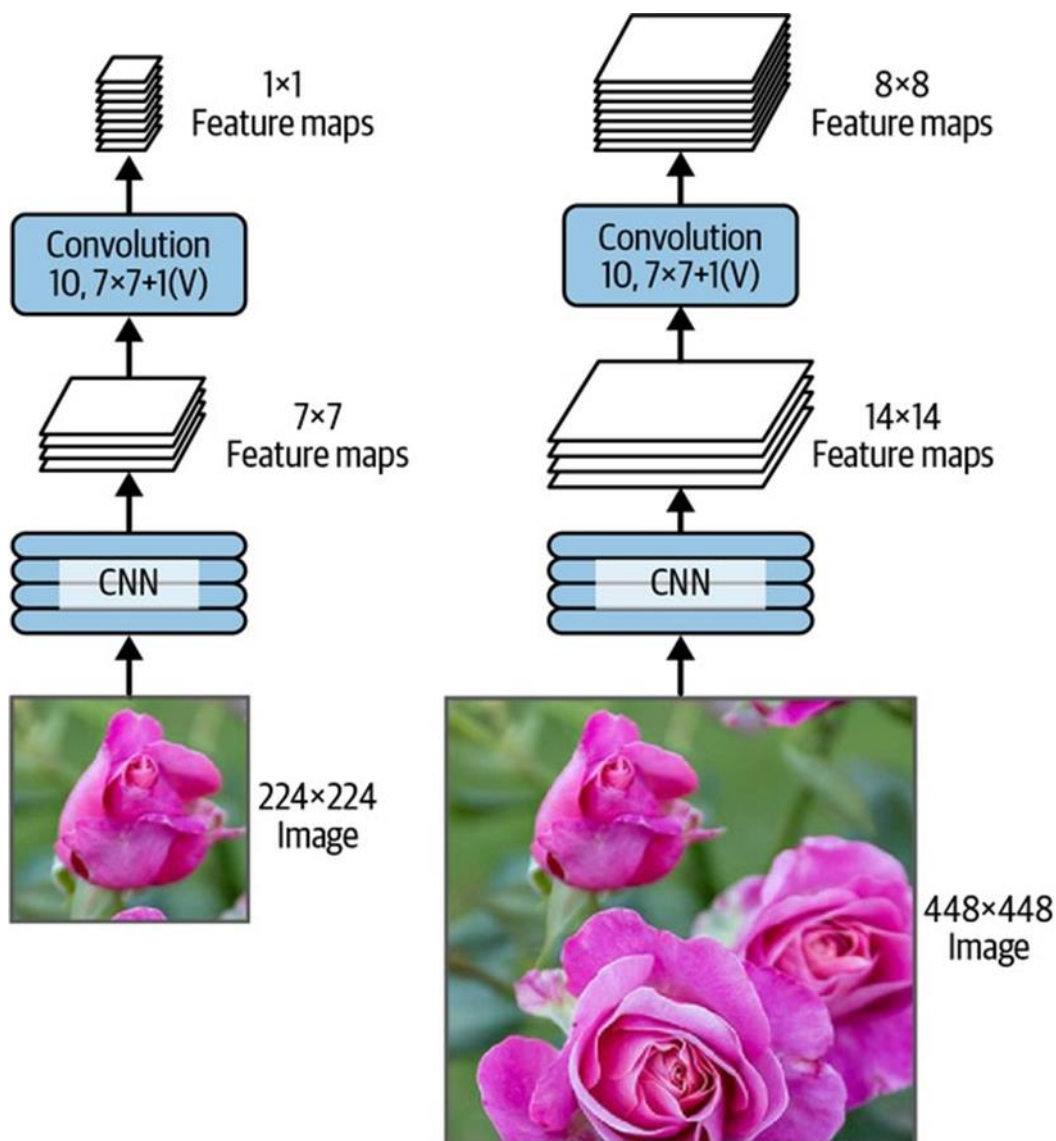
Ví dụ, giả sử chúng ta đã huấn luyện một CNN để phân loại và định vị hoa. Nó được huấn luyện trên hình ảnh 224×224 , và nó xuất ra 10 số:

- Đầu ra 0 đến 4 được gửi qua hàm kích hoạt softmax, và điều này cho các xác suất lớp (một trên mỗi lớp).
- Đầu ra 5 được gửi qua hàm kích hoạt sigmoid, và điều này cho điểm đối tượng.
- Đầu ra 6 và 7 đại diện cho tọa độ tâm của hộp bao quanh; chúng cũng đi qua hàm kích hoạt sigmoid để đảm bảo chúng nằm trong khoảng từ 0 đến 1.
- Cuối cùng, đầu ra 8 và 9 đại diện cho chiều cao và chiều rộng của hộp bao quanh; chúng không đi qua bất kỳ hàm kích hoạt nào để cho phép các hộp bao quanh mở rộng vượt ra ngoài ranh giới của hình ảnh.

Bây giờ chúng ta có thể chuyển đổi các lớp dày đặc của CNN thành các lớp tích chập. Trên thực tế, chúng ta thậm chí không cần huấn luyện lại nó; chúng ta chỉ cần sao chép các trọng số từ các lớp dày đặc sang các lớp tích chập! Hoặc, chúng ta có thể đã chuyển đổi CNN thành FCN trước khi huấn luyện.

Bây giờ giả sử lớp tích chập cuối cùng trước lớp đầu ra (còn gọi là lớp thắt cổ chai) xuất ra các bản đồ đặc trưng 7×7 khi mạng được cấp một hình ảnh 224×224 (xem phía bên trái của Hình 14-26). Nếu chúng ta cấp FCN một hình ảnh 448×448 (xem phía bên phải của Hình 14-26), lớp thắt cổ chai bây giờ sẽ xuất ra các bản đồ đặc trưng 14×14 . Vì lớp đầu ra dày đặc đã được thay thế bằng một lớp tích chập sử dụng 10 bộ lọc có kích thước 7×7 , với padding “valid” và stride 1, đầu ra sẽ bao gồm 10 bản đồ tính năng, mỗi bản đồ có kích thước 8×8 (vì $14 - 7 + 1 = 8$). Nói cách khác, FCN sẽ xử lý toàn bộ hình ảnh chỉ một lần, và nó sẽ xuất ra một lưới 8×8 trong đó mỗi ô chứa 10 số (5 xác suất lớp, 1 điểm đối tượng, và 4 tọa độ hộp bao quanh). Nó giống hệt như lấy CNN gốc và trượt nó trên hình ảnh bằng cách sử dụng 8 bước trên mỗi hàng và 8 bước trên mỗi cột. Để hình dung điều này, hãy tưởng tượng cắt hình ảnh gốc thành một lưới 14×14 , sau đó trượt một cửa sổ 7×7 trên lưới này; sẽ có $8 \times 8 = 64$ vị trí có thể cho cửa sổ, do đó có 8×8 dự đoán.

Tuy nhiên, cách tiếp cận FCN hiệu quả hơn nhiều, vì mạng chỉ nhìn vào hình ảnh một lần. Trên thực tế, You Only Look Once (YOLO) là tên của một kiến trúc phát hiện đối tượng rất phổ biến, chúng ta sẽ xem xét tiếp theo.



Hình 14-26. Cùng một mạng hoàn toàn tích chập xử lý một hình ảnh nhỏ (trái) và một hình ảnh lớn (phải).

You Only Look Once

YOLO là một kiến trúc phát hiện đối tượng nhanh và chính xác được Joseph Redmon và cộng sự đề xuất trong một bài báo năm 2015. Nó nhanh đến mức có thể chạy trong thời gian thực trên một video, như đã thấy trong bản demo của Redmon. Kiến trúc của YOLO khá giống với kiến trúc chúng ta vừa thảo luận, nhưng có một vài khác biệt quan trọng:

- Đối với mỗi ô lưới, YOLO chỉ xem xét các đối tượng có tâm hộp bao quanh nằm trong ô đó. Tọa độ hộp bao quanh tương đối so với ô đó, trong đó (0, 0) có nghĩa là góc trên bên trái của ô và (1, 1) có nghĩa là góc dưới bên phải. Tuy nhiên, chiều cao và chiều rộng của hộp bao quanh có thể mở rộng vượt ra ngoài ô.

- Nó xuất ra hai hộp bao quanh cho mỗi ô lưới (thay vì chỉ một), cho phép mô hình xử lý các trường hợp hai đối tượng quá gần nhau đến mức tâm hộp bao quanh của chúng nằm trong cùng một ô. Mỗi hộp bao quanh cũng đi kèm với điểm đối tượng riêng của nó.
- YOLO cũng xuất ra phân phối xác suất lớp cho mỗi ô lưới, dự đoán 20 xác suất lớp trên mỗi ô lưới vì YOLO được huấn luyện trên tập dữ liệu PASCAL VOC, chứa 20 lớp. Điều này tạo ra một bản đồ xác suất lớp thô. Lưu ý rằng mô hình dự đoán một phân phối xác suất lớp trên mỗi ô lưới, không phải trên mỗi hộp bao quanh. Tuy nhiên, có thể ước tính xác suất lớp cho mỗi hộp bao quanh trong quá trình xử lý hậu kỳ, bằng cách đo mức độ phù hợp của mỗi hộp bao quanh với mỗi lớp trong bản đồ xác suất lớp. Ví dụ, hãy tưởng tượng một bức ảnh một người đứng trước một chiếc ô tô. Sẽ có hai hộp bao quanh: một hộp lớn hình ngang cho chiếc ô tô, và một hộp nhỏ hơn hình dọc cho người. Các hộp bao quanh này có thể có tâm trong cùng một ô lưới. Vậy làm thế nào chúng ta có thể biết lớp nào nên được gán cho mỗi hộp bao quanh? Vâng, bản đồ xác suất lớp sẽ chứa một vùng lớn nơi lớp “car” chiếm ưu thế, và bên trong nó sẽ có một vùng nhỏ hơn nơi lớp “person” chiếm ưu thế. Hy vọng rằng, hộp bao quanh của chiếc ô tô sẽ khớp khoảng với vùng “car”, trong khi hộp bao quanh của người sẽ khớp khoảng với vùng “person”: điều này sẽ cho phép gán đúng lớp cho mỗi hộp bao quanh.

YOLO ban đầu được phát triển bằng Darknet, một framework học sâu mã nguồn mở ban đầu được Joseph Redmon phát triển bằng C, nhưng nó nhanh chóng được chuyển sang TensorFlow, Keras, PyTorch và nhiều hơn nữa. Nó liên tục được cải thiện trong những năm qua, với YOLOv2, YOLOv3 và YOLO9000 (cũng bởi Joseph Redmon và cộng sự), YOLOv4 (bởi Alexey Bochkovskiy và cộng sự), YOLOv5 (bởi Glenn Jocher) và PP-YOLO (bởi Xiang Long và cộng sự).

Mỗi phiên bản mang lại một số cải tiến ấn tượng về tốc độ và độ chính xác, sử dụng nhiều kỹ thuật khác nhau; ví dụ, YOLOv3 đã tăng cường độ chính xác một phần nhờ các neo tiên nghiệm (anchor priors), khai thác thực tế rằng một số hình dạng hộp bao quanh có nhiều khả năng hơn các hình dạng khác, tùy thuộc vào lớp (ví dụ: người có xu hướng có hộp bao quanh hình dọc, trong khi ô tô thường không). Họ cũng tăng số lượng hộp bao quanh trên mỗi ô lưới, họ huấn luyện trên các tập dữ liệu khác nhau với nhiều lớp hơn (lên đến 9.000 lớp được tổ chức theo hệ thống phân cấp trong trường hợp YOLO9000), họ thêm các kết nối bỏ qua để khôi phục một số độ phân giải không gian bị mất trong CNN (chúng ta sẽ thảo luận ngắn gọn về điều này, khi chúng ta xem xét phân đoạn ngữ nghĩa), và nhiều hơn nữa. Cũng có nhiều biến thể của các mô hình này, chẳng hạn như YOLOv4-tiny, được tối ưu hóa để được huấn luyện trên các máy kém mạnh hơn và có thể chạy cực nhanh (hơn 1.000 khung hình mỗi giây!), nhưng với độ chính xác trung bình (mAP) hơi thấp hơn.

Độ chính xác trung bình

Một thước đo rất phổ biến được sử dụng trong các tác vụ phát hiện đối tượng là độ chính xác trung bình (mean average precision). “Độ chính xác trung bình” nghe có vẻ hơi thừa, phải không? Để hiểu thước đo này, hãy quay lại hai thước đo phân loại chúng ta đã thảo

luận trong Chương 3: độ chính xác và độ thu hồi. Hãy nhớ sự đánh đổi: độ thu hồi càng cao, độ chính xác càng thấp. Bạn có thể hình dung điều này trong một đường cong độ chính xác/độ thu hồi (xem Hình 3-6). Để tóm tắt đường cong này thành một số duy nhất, chúng ta có thể tính diện tích dưới đường cong (AUC) của nó. Nhưng lưu ý rằng đường cong độ chính xác/độ thu hồi có thể chứa một vài phần mà độ chính xác thực sự tăng lên khi độ thu hồi tăng, đặc biệt ở các giá trị độ thu hồi thấp (bạn có thể thấy điều này ở phía trên bên trái của Hình 3-6). Đây là một trong những động lực cho thước đo mAP.

Giả sử bộ phân loại có độ chính xác 90% ở độ thu hồi 10%, nhưng độ chính xác 96% ở độ thu hồi 20%. Thực sự không có sự đánh đổi nào ở đây: đơn giản là có lý hơn khi sử dụng bộ phân loại ở độ thu hồi 20% hơn là ở độ thu hồi 10%, vì bạn sẽ nhận được cả độ thu hồi cao hơn và độ chính xác cao hơn. Vì vậy, thay vì nhìn vào độ chính xác ở độ thu hồi 10%, chúng ta nên thực sự nhìn vào độ chính xác tối đa mà bộ phân loại có thể cung cấp với ít nhất 10% độ thu hồi. Nó sẽ là 96%, không phải 90%.

Do đó, một cách để có được một ý tưởng công bằng về hiệu suất của mô hình là tính toán độ chính xác tối đa bạn có thể nhận được với ít nhất 0% độ thu hồi, sau đó 10% độ thu hồi, 20%, v.v. lên đến 100%, và sau đó tính trung bình các độ chính xác tối đa này. Điều này được gọi là thước đo độ chính xác trung bình (AP). Bây giờ khi có nhiều hơn hai lớp, chúng ta có thể tính AP cho mỗi lớp, và sau đó tính AP trung bình (mAP). Thế thôi!

Trong một hệ thống phát hiện đối tượng, có một mức độ phức tạp bổ sung: điều gì sẽ xảy ra nếu hệ thống phát hiện đúng lớp, nhưng ở sai vị trí (tức là hộp bao quanh hoàn toàn sai)? Chắc chắn chúng ta không nên coi đây là một dự đoán tích cực. Một cách tiếp cận là định nghĩa một ngưỡng IoU: ví dụ, chúng ta có thể coi một dự đoán là đúng chỉ khi IoU lớn hơn, chẳng hạn, 0,5, và lớp được dự đoán là đúng. mAP tương ứng thường được ký hiệu là $\text{mAP}@0.5$ (hoặc $\text{mAP}@50\%$, hoặc đôi khi chỉ $\text{AP}50$). Trong một số cuộc thi (chẳng hạn như thử thách PASCAL VOC), đây là điều được thực hiện. Trong các cuộc thi khác (chẳng hạn như cuộc thi COCO), mAP được tính toán cho các ngưỡng IoU khác nhau (0.50, 0.55, 0.60, ..., 0.95), và thước đo cuối cùng là trung bình của tất cả các mAP này (được ký hiệu là $\text{mAP}@[.50:.95]$ hoặc $\text{mAP}@[.50:0.05:.95]$). Vâng, đó là một trung bình của trung bình.

Nhiều mô hình phát hiện đối tượng có sẵn trên TensorFlow Hub, thường với các trọng số đã được huấn luyện sẵn, chẳng hạn như YOLOv5, SSD, Faster R-CNN và EfficientDet. SSD và EfficientDet là các mô hình phát hiện “nhìn một lần”, tương tự như YOLO. EfficientDet dựa trên kiến trúc tích chập EfficientNet. Faster R-CNN phức tạp hơn: hình ảnh đầu tiên đi qua một CNN, sau đó đầu ra được truyền đến một mạng đề xuất vùng (RPN) đề xuất các hộp bao quanh có nhiều khả năng chứa một đối tượng; một bộ phân loại sau đó được chạy cho mỗi hộp bao quanh, dựa trên đầu ra đã cắt của CNN. Nơi tốt nhất để bắt đầu sử dụng các mô hình này là hướng dẫn phát hiện đối tượng tuyệt vời của TensorFlow Hub.

Cho đến nay, chúng ta chỉ xem xét việc phát hiện đối tượng trong các hình ảnh đơn lẻ. Nhưng còn video thì sao? Các đối tượng không chỉ phải được phát hiện trong mỗi khung hình, mà chúng còn phải được theo dõi theo thời gian. Hãy cùng xem nhanh về theo dõi đối tượng ngay bây giờ.

Theo dõi đối tượng

Theo dõi đối tượng là một nhiệm vụ đầy thách thức: các đối tượng di chuyển, chúng có thể lớn lên hoặc co lại khi đến gần hoặc di chuyển ra xa camera, hình dáng của chúng có thể thay đổi khi chúng quay hoặc di chuyển đến các điều kiện ánh sáng hoặc nền khác nhau, chúng có thể tạm thời bị che khuất bởi các đối tượng khác, v.v.

Một trong những hệ thống theo dõi đối tượng phổ biến nhất là DeepSORT. Nó dựa trên sự kết hợp giữa các thuật toán cổ điển và học sâu:

- Nó sử dụng các bộ lọc Kalman để ước tính vị trí hiện tại có khả năng nhất của một đối tượng dựa trên các phát hiện trước đó, và giả định rằng các đối tượng có xu hướng di chuyển với tốc độ không đổi.
- Nó sử dụng một mô hình học sâu để đo lường sự tương đồng giữa các phát hiện mới và các đối tượng đã được theo dõi.
- Cuối cùng, nó sử dụng thuật toán Hungary để ánh xạ các phát hiện mới vào các đối tượng đã được theo dõi (hoặc vào các đối tượng được theo dõi mới): thuật toán này tìm kiếm hiệu quả sự kết hợp các ánh xạ giúp giảm thiểu khoảng cách giữa các phát hiện và vị trí dự đoán của các đối tượng được theo dõi, đồng thời giảm thiểu sự khác biệt về hình dáng.

Ví dụ, hãy tưởng tượng một quả bóng đỏ vừa bật ra khỏi một quả bóng xanh lam đang di chuyển theo hướng ngược lại. Dựa trên các vị trí trước đó của các quả bóng, bộ lọc Kalman sẽ dự đoán rằng các quả bóng sẽ đi xuyên qua nhau: thực tế, nó giả định rằng các đối tượng di chuyển với tốc độ không đổi, vì vậy nó sẽ không mong đợi cú bật. Nếu thuật toán Hungary chỉ xem xét các vị trí, thì nó sẽ vui vẻ ánh xạ các phát hiện mới vào các quả bóng sai, như thể chúng vừa đi xuyên qua nhau và đổi màu. Nhưng nhờ vào thước đo sự tương đồng, thuật toán Hungary sẽ nhận thấy vấn đề. Giả sử các quả bóng không quá giống nhau, thuật toán sẽ ánh xạ các phát hiện mới vào các quả bóng đúng.

Cho đến nay, chúng ta đã định vị các đối tượng bằng cách sử dụng các hộp bao quanh. Điều này thường là đủ, nhưng đôi khi bạn cần định vị các đối tượng với độ chính xác cao hơn nhiều—ví dụ, để loại bỏ nền phía sau một người trong cuộc gọi hội nghị truyền hình. Hãy xem cách đi xuống cấp độ pixel.

Phân đoạn ngữ nghĩa

Trong phân đoạn ngữ nghĩa, mỗi pixel được phân loại theo lớp của đối tượng mà nó thuộc về (ví dụ: đường, ô tô, người đi bộ, tòa nhà, v.v.), như thể hiện trong Hình 14-27. Lưu ý rằng các đối tượng khác nhau cùng lớp không được phân biệt. Ví dụ, tất cả các chiếc xe đạp ở phía bên phải của hình ảnh đã phân đoạn cuối cùng trở thành một khối pixel lớn. Khó khăn chính trong nhiệm vụ này là khi hình ảnh đi qua một CNN thông thường, chúng dần dần mất độ phân giải không gian (do các lớp có stride lớn hơn 1); do đó, một CNN thông thường có thể biết rằng có một người ở đâu đó ở phía dưới bên trái của hình ảnh, nhưng nó sẽ không chính xác hơn thế.

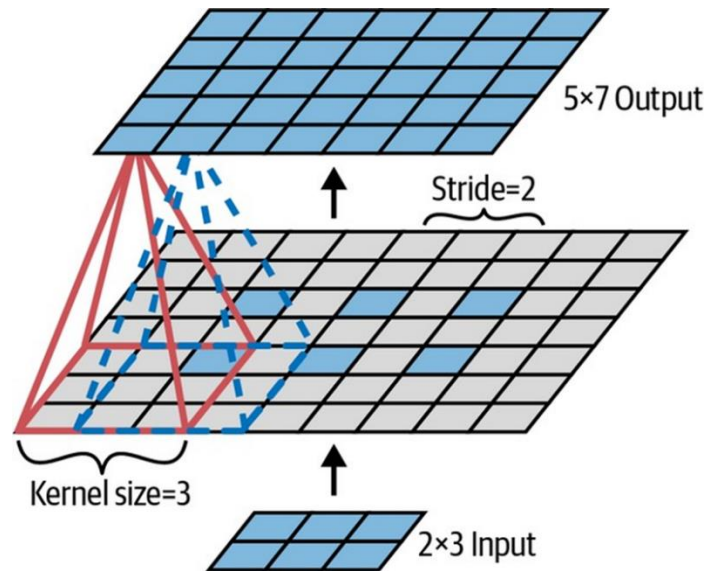


Hình 14-27. Phân đoạn ngữ nghĩa.

Giống như đối với phát hiện đối tượng, có nhiều cách tiếp cận khác nhau để giải quyết vấn đề này, một số khá phức tạp. Tuy nhiên, một giải pháp khá đơn giản đã được đề xuất trong bài báo năm 2015 của Jonathan Long et al. mà tôi đã đề cập trước đó, về mạng hoàn toàn tích chập. Các tác giả bắt đầu bằng cách lấy một CNN đã được huấn luyện sẵn và biến nó thành một FCN. CNN áp dụng một stride tổng thể là 32 cho hình ảnh đầu vào (tức là, nếu bạn cộng tất cả các stride lớn hơn 1), có nghĩa là lớp cuối cùng xuất ra các bản đồ đặc trưng nhỏ hơn 32 lần so với hình ảnh đầu vào. Điều này rõ ràng là quá thô, vì vậy họ đã thêm một lớp lấy mẫu lên (upsampling layer) nhân độ phân giải lên 32.

Có một số giải pháp có sẵn để lấy mẫu lên (tăng kích thước của hình ảnh), chẳng hạn như nội suy song tuyến, nhưng điều đó chỉ hoạt động khá tốt lên đến $\times 4$ hoặc $\times 8$. Thay vào đó, họ sử dụng một lớp tích chập chuyển vị (transposed convolutional layer): điều này tương đương với việc đầu tiên kéo giãn hình ảnh bằng cách chèn các hàng và cột trống (đầy số 0), sau đó thực hiện một tích chập thông thường (xem Hình 14-28). Hoặc, một số người thích nghĩ nó như một lớp tích chập thông thường sử dụng các stride phân số (ví dụ: stride là $1/2$ trong Hình 14-28).

Lớp tích chập chuyển vị có thể được khởi tạo để thực hiện một cái gì đó gần với nội suy tuyến tính, nhưng vì nó là một lớp có thể huấn luyện được, nó sẽ học cách làm tốt hơn trong quá trình huấn luyện. Trong Keras, bạn có thể sử dụng lớp Conv2DTranspose.



Hình 14-28. Lấy mẫu lên bằng cách sử dụng một lớp tích chập chuyển vị.

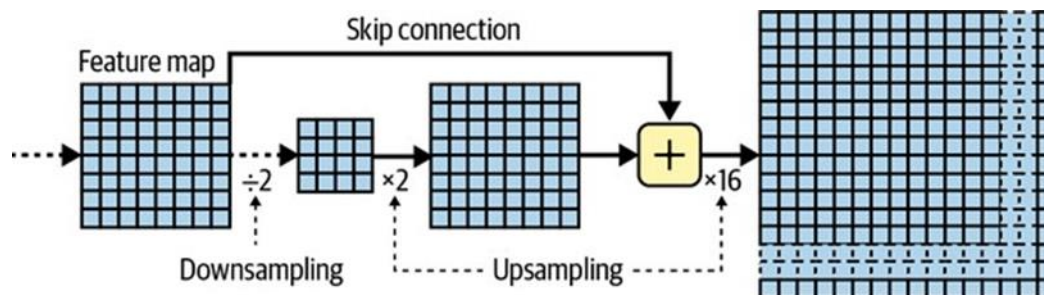
Các lớp tích chập Keras khác

Keras cũng cung cấp một vài loại lớp tích chập khác:

- `tf.keras.layers.Conv1D`: Một lớp tích chập cho đầu vào 1D, chẳng hạn như chuỗi thời gian hoặc văn bản (chuỗi chữ cái hoặc từ), như bạn sẽ thấy trong Chương 15.
- `tf.keras.layers.Conv3D`: Một lớp tích chập cho đầu vào 3D, chẳng hạn như quét PET 3D.
- `dilation_rate`: Đặt siêu tham số `dilation_rate` của bất kỳ lớp tích chập nào thành giá trị từ 2 trở lên sẽ tạo ra một lớp tích chập à-trous (“à trous” trong tiếng Pháp có nghĩa là “có lỗ”). Điều này tương đương với việc sử dụng một lớp tích chập thông thường với một bộ lọc được giãn ra bằng cách chèn các hàng và cột số 0 (tức là các lỗ). Ví dụ, một bộ lọc 1×3 bằng `[[1,2,3]]` có thể được giãn ra với tốc độ giãn nở là 4, dẫn đến một bộ lọc giãn nở `[[1, 0, 0, 0, 2, 0, 0, 0, 3]]`. Điều này cho phép lớp tích chập có trường tiếp nhận lớn hơn mà không tốn chi phí tính toán và không sử dụng thêm tham số.

Sử dụng các lớp tích chập chuyển vị để lấy mẫu lên là được, nhưng vẫn quá thiếu chính xác. Để làm tốt hơn, Long et al. đã thêm các kết nối bỏ qua từ các lớp thấp hơn: ví dụ, họ lấy mẫu lên hình ảnh đầu ra theo hệ số 2 (thay vì 32), và họ thêm đầu ra của một lớp thấp hơn có độ phân giải gấp đôi này. Sau đó, họ lấy mẫu lên kết quả theo hệ số 16, dẫn đến tổng hệ số lấy mẫu lên là 32 (xem Hình 14-29). Điều này đã khôi phục một số độ phân giải không gian bị mất trong các lớp gộp trước đó. Trong kiến trúc tốt nhất của họ, họ đã sử dụng một kết nối bỏ qua tương tự thứ hai để khôi phục các chi tiết thậm chí còn tốt hơn từ một lớp thậm chí còn thấp hơn. Tóm lại, đầu ra của CNN gốc đi qua các bước bổ sung sau: lấy mẫu lên $\times 2$, thêm đầu ra của một lớp thấp hơn (ở tỷ lệ thích hợp), lấy mẫu lên $\times 2$, thêm đầu ra của một lớp thậm chí còn thấp hơn, và cuối cùng lấy mẫu lên $\times 8$. Thậm chí có thể

mở rộng vượt quá kích thước của hình ảnh gốc: điều này có thể được sử dụng để tăng độ phân giải của một hình ảnh, đây là một kỹ thuật gọi là siêu phân giải (super-resolution).



Hình 14-29. Các lớp bỏ qua khôi phục một số độ phân giải không gian từ các lớp thấp hơn.

Phân đoạn thể hiện (Instance segmentation) tương tự như phân đoạn ngữ nghĩa, nhưng thay vì hợp nhất tất cả các đối tượng cùng lớp thành một khối lớn, mỗi đối tượng được phân biệt với các đối tượng khác (ví dụ: nó xác định từng chiếc xe đạp riêng lẻ). Ví dụ, kiến trúc Mask R-CNN, được đề xuất trong một bài báo năm 2017 bởi Kaiming He et al., mở rộng mô hình Faster R-CNN bằng cách tạo thêm một mặt nạ pixel cho mỗi hộp bao quanh. Vì vậy, bạn không chỉ nhận được một hộp bao quanh xung quanh mỗi đối tượng, với một tập hợp các xác suất lớp ước tính, mà bạn còn nhận được một mặt nạ pixel định vị các pixel trong hộp bao quanh thuộc về đối tượng. Mô hình này có sẵn trên TensorFlow Hub, đã được huấn luyện sẵn trên tập dữ liệu COCO 2017. Lĩnh vực này đang phát triển nhanh chóng, vì vậy nếu bạn muốn thử các mô hình mới nhất và tốt nhất, vui lòng kiểm tra phần state-of-the-art của <https://paperswithcode.com>.

Như bạn có thể thấy, lĩnh vực thị giác máy tính sâu rộng lớn và phát triển nhanh chóng, với tất cả các loại kiến trúc xuất hiện hàng năm. Hầu hết trong số đó đều dựa trên mạng nơ-ron tích chập, nhưng kể từ năm 2020, một kiến trúc mạng nơ-ron khác đã gia nhập không gian thị giác máy tính: transformers (mà chúng ta sẽ thảo luận trong Chương 16). Sự tiến bộ đạt được trong thập kỷ qua là đáng kinh ngạc, và các nhà nghiên cứu hiện đang tập trung vào các vấn đề ngày càng khó hơn, chẳng hạn như học đối nghịch (cố gắng làm cho mạng có khả năng chống lại các hình ảnh được thiết kế để đánh lừa nó), khả năng giải thích (hiểu tại sao mạng đưa ra một phân loại cụ thể), tạo hình ảnh thực tế (chúng ta sẽ quay lại vấn đề này trong Chương 17), học một lần (một hệ thống có thể nhận dạng một đối tượng sau khi nó chỉ nhìn thấy nó một lần), dự đoán các khung hình tiếp theo trong một video, kết hợp các tác vụ văn bản và hình ảnh, và nhiều hơn nữa.

Bây giờ sang chương tiếp theo, chúng ta sẽ xem xét cách xử lý dữ liệu tuần tự như chuỗi thời gian bằng cách sử dụng mạng nơ-ron hồi quy và mạng nơ-ron tích chập.

Bài tập

1. Ưu điểm của CNN so với DNN kết nối đầy đủ cho phân loại hình ảnh là gì?
2. Xem xét một CNN gồm ba lớp tích chập, mỗi lớp có kernel 3×3 , stride 2 và padding “same”. Lớp thấp nhất xuất ra 100 bản đồ đặc trưng, lớp giữa xuất ra 200, và lớp trên cùng xuất ra 400. Các hình ảnh đầu vào là hình ảnh RGB có kích thước $200 \times$

300 pixel: a. Tổng số tham số trong CNN là bao nhiêu? b. Nếu chúng ta sử dụng số dấu phẩy động 32 bit, mạng này sẽ yêu cầu ít nhất bao nhiêu RAM khi đưa ra dự đoán cho một trường hợp duy nhất? c. Còn khi huấn luyện trên một mini-batch gồm 50 hình ảnh thì sao?

3. Nếu GPU của bạn hết bộ nhớ khi huấn luyện CNN, bạn có thể thử năm điều gì để giải quyết vấn đề?
4. Tại sao bạn muốn thêm một lớp gộp max thay vì một lớp tích chập có cùng stride?
5. Khi nào bạn muốn thêm một lớp chuẩn hóa phản hồi cục bộ?
6. Bạn có thể nêu tên những đổi mới chính trong AlexNet, so với LeNet-5 không? Còn những đổi mới chính trong GoogLeNet, ResNet, SEnet, Xception và EfficientNet thì sao?
7. Mạng hoàn toàn tích chập là gì? Làm thế nào bạn có thể chuyển đổi một lớp dày đặc thành một lớp tích chập?
8. Khó khăn kỹ thuật chính của phân đoạn ngữ nghĩa là gì?
9. Xây dựng CNN của riêng bạn từ đầu và cố gắng đạt được độ chính xác cao nhất có thể trên MNIST.
10. Sử dụng học chuyển đổi cho phân loại hình ảnh lớn, trải qua các bước sau: a. Tạo một tập huấn luyện chứa ít nhất 100 hình ảnh trên mỗi lớp. Ví dụ, bạn có thể phân loại hình ảnh của riêng mình dựa trên vị trí (bãi biển, núi, thành phố, v.v.), hoặc thay vào đó bạn có thể sử dụng một tập dữ liệu hiện có (ví dụ: từ TensorFlow Datasets). b. Chia nó thành tập huấn luyện, tập xác thực và tập kiểm tra. c. Xây dựng đường ống đầu vào, áp dụng các thao tác tiền xử lý thích hợp và tùy chọn thêm tăng cường dữ liệu. d. Tinh chỉnh một mô hình đã được huấn luyện sẵn trên tập dữ liệu này.
11. Đọc qua hướng dẫn Chuyển đổi phong cách của TensorFlow. Đây là một cách thú vị để tạo ra nghệ thuật bằng cách sử dụng học sâu. Giải pháp cho các bài tập này có sẵn ở cuối sổ tay của chương này, tại <https://homl.info/colab3>.